

Министерство образования Республики Беларусь

Учреждение образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 Программная инженерия.
Профилизация: Программное обеспечение информационных технологий

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
КУРСОВОГО ПРОЕКТА НА ТЕМУ:**

«Разработка компилятора LPA-2025»

Выполнил студент Лебедь Полина Андреевна
курс, группа _____ подпись _____ (Ф.И.О. студента)

Руководитель проекта преп. стажер, Авдеева В. Д.
(учен. степень, звание, должность, подпись, Ф.И.О. руководителя)

Заведующий кафедрой к.т.н., доц. Смелов В.В.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Консультанты преп. стажер, Авдеева В. Д.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой _____

Минск 2025

Содержание

Введение.....	5
1. Спецификация языка программирования.....	6
1.1 Характеристика языка программирования.....	6
1.2 Определение алфавита языка программирования.....	6
1.3 Применяемые сепараторы.....	6
1.4 Применяемые кодировки.....	7
1.5 Типы данных	8
1.6 Преобразование типов данных	8
1.7 Идентификаторы	9
1.8 Литералы.....	9
1.9 Объявление данных	10
1.10 Инициализация данных.....	10
1.11 Инструкции языка	11
1.12 Операции языка.....	11
1.13 Выражения и их вычисления	12
1.14 Конструкции языка	12
1.15 Область видимости идентификаторов	13
1.16 Семантические проверки	13
1.17 Распределение оперативной памяти на этапе выполнения	14
1.18 Стандартная библиотека и ее состав.....	14
1.19 Ввод и вывод данных.....	15
1.20 Точка входа	15
1.21 Препроцессор	15
1.22 Соглашения о вызовах	16
1.23 Объектный код.....	16
1.24 Классификация сообщений транслятора.....	16
1.25 Контрольный пример.....	16
2. Структура транслятора	17
2.1. Компоненты транслятора их назначение и принципы взаимодействия	17
2.2. Параметры транслятора, управляющие его работой.....	18
2.3 Протоколы, формируемые транслятором	18
3. Разработка лексического анализатора.....	20
3.1. Структура лексического анализатора.....	20

3.2. Контроль входных символов.....	20
3.3. Удаление избыточных символов	21
3.4. Перечень ключевых слов.....	22
3.5. Основные структуры данных.....	23
3.6. Структура и перечень сообщений лексического анализатора.....	24
3.7. Принцип обработки ошибок	25
3.8. Параметры лексического анализатора	25
3.9. Алгоритм лексического анализа.....	26
3.10. Контрольный пример.....	27
4. Разработка синтаксического анализатора.....	28
4.1. Структура синтаксического анализатора.....	28
4.2. Контекстно-свободная грамматика, описывающая синтаксис языка	28
4.3. Построение конечного магазинного автомата	30
4.4. Основные структуры данных.....	31
4.5. Описание алгоритма синтаксического разбора	32
4.6. Структура и перечень сообщений синтаксического анализатора.....	32
4.7. Параметры синтаксического анализатора и режимы его работы	33
4.8. Принцип обработки ошибок	33
4.9. Контрольный пример.....	33
5. Разработка семантического анализатора	34
5.1. Структура семантического анализатора	34
5.2. Функции семантического анализатора	34
5.3. Структура анализатора	35
5.4. Принцип обработки ошибок	36
5.5. Контрольный пример.....	36
6. Вычисление выражений	37
6.1. Выражения, допускаемые языком	37
6.2. Польская запись и принцип ее построения	37
6.3. Программная реализация обработки выражений	38
6.4. Контрольный пример.....	38
7. Генерация кода.....	39
7.1. Структура генератора кода	39
7.2. Представление типов данных в оперативной памяти	39
7.3. Статическая библиотека	40

7.4. Особенности алгоритма генерации кода	40
7.5. Входные параметры, управляющие генерацией кода	41
7.6. Контрольный пример.....	41
8. Тестирование транслятора	42
8.1. Общие положения	42
8.2. Результаты тестирования.....	42
Заключение	44
Список используемых источников	45
Приложение А	46
Приложение Б.....	48
Приложение В.....	49
Приложение Г	53
Приложение Д.....	54
Приложение Е.....	58
Приложение Ж.....	61

Введение

Целью данного курсового проекта является разработка языка программирования LPA-2025 и компилятора для его трансляции. Проект нацелен на изучение и практическое применение принципов создания языка программирования и транслятора. Разработка компилятора будет выполняться на языке C++, а результатом его работы будет генерация кода на языке ассемблера.

Компилятор состоит из следующих частей:

- лексический анализатор;
- синтаксический анализатор;
- семантический анализатор;
- генератор кода.

Для разработки компилятора необходимо выполнить следующие задачи:

- разработать спецификацию языка программирования LPA-2025, включая описание его синтаксиса и семантики;
- определить структуру транслятора, описав его основные компоненты и их взаимодействие;
- разработать структуры основных анализаторов: лексического, синтаксического и семантического;
- реализовать процесс генерации объектного кода, демонстрирующий функциональные возможности языка;
- создать контрольный пример программы, написанной на языке LPA-2025, для проверки и демонстрации работоспособности транслятора;
- провести тестирование транслятора с целью выявления ошибок и подтверждения корректности его работы.

Решения каждой из поставленных целей будут приведены в соответствующих разделах курсового проекта.

1. Спецификация языка программирования

1.1 Характеристика языка программирования

Язык программирования LPA-2025 является процедурным, универсальным, строго типизированным, не объектно-ориентированным, компилируемым.

1.2 Определение алфавита языка программирования

Язык программирования LPA-2025 использует стандартную кодировку символов Windows-1251. Его структура представлена в форме Бэкуса-Наура в таблице 1.1

Таблица 1.1 – Алфавит языка программирования LPA-2025

<строчная буква латинского алфавита>::= a b c d e f g h i j k l m n o p q r s t u v w x y z
<прописная буква латинского алфавита>::= A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
<цифра>::= 0 1 2 3 4 5 6 7 8 9
<символ-сепаратор>::= пробел , () { } ; “ ” ‘ ’
<операции сравнения>::= > < == <= >=
<унарные операции>::= ++ -- ~
<арифметические операции>::= + - *

Таким образом, алфавит языка строго ограничен представленным набором. Лексический анализатор языка LPA-2025 будет распознавать и обрабатывать лишь символы, входящие в эти категории. Любой иной символ кодировки Windows-1251, встреченный в исходном коде, приведёт к ошибке на этапе лексического анализа.

1.3 Применяемые сепараторы

Символы-сепараторы необходимы для эффективного разбиения исходного текста программы на лексемы на этапе лексического анализа. В таблице 1.2 приведены сепараторы и их назначение.

Таблица 1.2 – Применяемые сепараторы

Сепаратор	Назначение
1	2
пробел	Разделитель лексем. Не допускается в ключевых словах и идентификаторах
;(точка с запятой)	Разделитель строк
{ }	Определитель блока функций
()	Определитель параметров функций
, (запятая)	Разделитель параметров функции
++ -- ~	Выполнение унарных операций
=	Оператор присвоения

Продолжение таблицы 1.2

1	2
> <	Операции сравнения
+ - * /	Арифметические операции
&	Побитовые операции
“ ”	Строковый литерал
‘ ’	Символьный литерал

Набор сепараторов определяет структуру кода, задавая правила разделения элементов. Их правильное использование обеспечивает корректное выделение лексем и интерпретацию программы.

1.4 Применяемые кодировки

Для написания исходного кода программы на языке LPA-2025 используется кодировка Windows-1251. Символы, поддерживаемые в кодировке Windows-1251 представлены на рисунке 1.1.

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	NUL	STX	SOT	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
10	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
20	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
30	0	1	2	3	4	5	6	7	8	9	:	<	=	>	?	
40	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
60	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
80	Ђ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ
90	ђ	ѓ	ѕ	ї	љ	њ	ћ	ќ	ѐ	‰	љ	>	њ	ќ	ћ	џ
A0	Њ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ
B0	Њ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ
C0	Њ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ
D0	Њ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ
E0	Њ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ
F0	Њ	Ѓ	Ѕ	Ї	Љ	Њ	Ћ	Ќ	Є	‰	Љ	<	Њ	Ќ	Ќ	Ќ

Рисунок 1.1 – Таблица кодировок Windows-1251

Windows-1251 является стандартной и единственной допустимой кодировкой для исходного кода LPA-2025. Это обеспечивает однозначное соответствие между байтами в файле и символами алфавита языка. Использование любой другой кодировки приведёт к ошибке компиляции.

1.5 Типы данных

Язык программирования LPA-2025 поддерживает использование следующих типов данных: целочисленный (1 байт), целочисленный (4 байта), символьный, строковый, логический. Описание типов данных представлено в таблице 1.3.

Таблица 1.3 – Таблица данных

Тип данных	Описание
1	2
Целочисленный (1 байт) byte	Целочисленный тип данных. Занимает 1 байт. Диапазон значений: от -128 до 127. Возможные операции: арифметические (+, -, *); логические (&,); унарные (++ , --, ~); операции сравнения (>, <) Инициализация по умолчанию: 0.
Символьный symb	Символьный тип данных. Занимает 1 байт. Диапазон значений: все символы кодировки Windows-1251. Инициализация по умолчанию: нуль-символ ('\0').
Строковый str	Строковый тип данных. Используется для работы с набором символов, каждый из которых занимает 1 байт. Диапазон значений: все символы кодировки Windows-1251. Инициализация по умолчанию: нуль-символ ('\0').
Целочисленный (4 байта) int	Целочисленный тип данных. Занимает 4 байта. Диапазон значений: от -2^{16} до $2^{16} - 1$. Возможные операции: арифметические (+, - *); логические (&,); операции сравнения (>, <); унарные операции(++ , --, ~) Инициализация по умолчанию: 0
Логический torf	Логический тип данных. Занимает 1 байт, значение 0 или по или любое отличное от 0 или yes. Инициализация по умолчанию: 0.

Этого набора достаточно для решения типовых задач обработки целочисленных, символьных, строковых и логических данных. Все переменные должны быть объявлены с указанием одного из этих типов перед их использованием.

1.6 Преобразование типов данных

В языке программирования LPA-2025 преобразование типов данных не поддерживается, все типы данных определены однозначно. Но в стандартной библиотеке есть функция преобразования типа str в тип int.

1.7 Идентификаторы

Идентификаторы в языке LPA-2025 используются для именования функций, процедур, параметров функций и переменных. Идентификаторы, объявленные внутри блока функции или процедуры получают префикс, идентичный имени функции/процедуры для разрешения области видимости.

Идентификаторы не могут содержать символов кириллицы, специальных символов; не могут начинаться с цифры. Также идентификаторы не должны совпадать с ключевыми словами языка.

Максимальная длина идентификатора не должна превышать 9 символов.

Формальное описание идентификатора приведено на рисунке 1.2.

```
<идентификатор> ::= <буква> { <буква> | <цифра> } >
<буква> ::=
a|b|c|d|e|f|g|h|i|j|k|l|m|n|o|p|q|r|s|t|u|v|w|x|y|z|A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|
Y|Z
<цифра> ::= 0|1|2|3|4|5|6|7|8|9
```

Рисунок 1.2 – Формальное описание идентификатора

Примеры правильных идентификаторов: mySymb, count, a.

Примеры неправильных идентификаторов: 0var, byte, a+.

1.8 Литералы

Литералы языка программирования – последовательность символов, используемая для именования неизменяемых значений в коде.

В языке LPA-2025 предусмотрены следующие типы литералов: целочисленный, символьный. Описание литералов представлено в таблице 1.4.

Таблица 1.4 – Литералы LPA-2025

Литерал	Описание	Примечание
Целочисленный	Задается в двоичном или десятичном представлениях	Двоичный литерал: [0-1] ⁺ b Десятичный литерал: [0-9] ⁺ d
Символьный	Один разрешенный символ	Заключается в одинарные кавычки ' '
Строковый	Последовательность символов. Длина может варьироваться в пределах от 1 до 255.	Заключается в двойные кавычки " "

Пример правильных литералов: "Yes", 'a', 1011b, 123d, 123.

Пример неправильных литералов: 'No', "n", 432b.

1.9 Объявление данных

При объявлении переменных в языке LPA-2025 необходимо учитывать следующие правила: необходимо объявлять переменную до её использования; при объявлении используется ключевое слово **def**; переменная может быть объявлена в любом программном блоке, если она уже не была объявлена в данной области видимости; допускается инициализация переменной при объявлении.

Объявление функции схоже с объявлением обычной переменной: сначала указывается возвращаемый функцией тип данных, разрешенный в языке программирования; потом используется ключевое слово **task**; далее определяется имя функции; в круглых скобках указываются параметры (скобки остаются пустыми, если параметры не передаются); затем в фигурных скобках определяется блок функции; в блоке функции используется ключевое слово **response** для возврата значения.

1.10 Инициализация данных

В языке программирования LPA-2025 допускается инициализация данных при объявлении переменной. При этом переменной будет присвоено значение литерала или идентификатора, стоящего справа от знака равенства. Объектами-инициализаторами могут быть идентификаторы, литералы, выражения и вызовы функций. Предусмотрены значения по умолчанию, если переменные не инициализированы. Способы инициализации переменных в языке программирования LPA-2025 представлены в таблице 1.5

Таблица 1.5 – Способы инициализации переменных

Вид инициализации	Примечание
<code>def <тип данных> <идентификатор>;</code>	Инициализация по умолчанию при объявлении: переменные типа <code>byte</code> инициализируются нулём; переменные типа <code>str</code> – пустой строкой, ограниченной двойными кавычками, переменные типа <code>symb</code> – пустой символом, ограниченным одинарными кавычками.
<code>def <тип данных> <идентификатор> = <значение> <выражение>;</code> <code><значение>::= <литерал> </code> <code><идентификатор> <вызов функции></code>	Присваивание значения идентификатора, литерала, результата вычисления выражения или функции при объявлении.
<code><идентификатор> = <значение> </code> <code><выражение>;</code> <code><значение>::= <литерал> </code> <code><идентификатор> <вызов функции></code>	Присваивание значения при повторном использовании.

Таким образом, LPA-2025 предлагает гибкие способы инициализации: можно явно задавать значения при объявлении или полагаться на стандартные значения по умолчанию.

1.11 Инструкции языка

Инструкции языка программирования LPA-2025 представлены в таблице 1.6.

Таблица 1.6 – Инструкции языка LPA-2025

Инструкция	Описание	Пример
Объявление переменной	def <тип данных> <идентификатор>;	def byte number;
Объявление переменной с инициализацией	def <тип данных> <идентификатор> = <значение> <выражение>; <значение>::= <литерал> <идентификатор> <вызов функции>;	def byte number = 1+6;
Объявление функции	<тип данных> task <идентификатор функции> (<список параметров>) {<блок кода>;}	byte task sum(a, b) {... response ...};
Вызов функции	<идентификатор функции> (<параметры функции> <пусто>);	sum(a,b); writeText();
Присваивание	<идентификатор> = <литерал>; <идентификатор> = <выражение>; <идентификатор1> = <идентификатор2>;	number = 3; number = 10 – 9; number = number2;
Возврат из подпрограммы	response <идентификатор> <литерал>;	response number;
Вывод данных	cout (<идентификатор> <литерал>);	cout (number);
Условный оператор с выбором из двух значений	when(<условное выражение>) {<тело блока истины>} other {<тело блока лжи>}	when(number < 3) { cout(1); } other { cout(5);}
Условный оператор с выбором из множества значений	switch(<выражение>){ is<значение_1>:{<тело_1>;} is<значение_2>:{<тело_2>;} any:{<тело_по_умолчанию>;} }	switch(number){ is 2:{cout(“two”);} is 7:{cout(“seven”);} any:{cout(“other”);} }

Базовый набор инструкций LPA-2025 позволяет выполнять все фундаментальные операции программирования: объявлять данные, управлять логикой выполнения и организовывать код в подпрограммы. Этот минимальный, но полный набор конструкций является основой для построения любых корректных программ на языке.

1.12 Операции языка

Язык программирования LPA-2025 поддерживает различные операции. Приоритетность операций определяется с помощью (). 1 – наивысшая приоритетность, 4 – низшая. Операции представлены в таблице 1.7.

Таблица 1.7 – Операции языка

Оператор	Назначение	Приоритетность
()	Приоритетность операций	0
++, --	Унарный инкремент, декремент	4
~	Инверсия	4
*	Умножение	3
+, -	Сложение, вычитание	2
<, >	Операции сравнения	1
&,	Битовые операции	1
,	Запятая	1

Порядок выполнения операций в LPA-2025 строго определяется указанными приоритетами. Операции с более высоким приоритетом (меньшим числовым значением) обрабатываются в первую очередь. Для явного управления последовательностью вычислений следует использовать круглые скобки – это делает код нагляднее и предотвращает ошибки, связанные с неявным порядком операций.

1.13 Выражения и их вычисления

Предусмотрены следующие правила составления выражений:

- рассматриваются слева направо;
- для изменения приоритета операции используются круглые скобки ();
- каждое выражение должно заканчиваться сепаратором;
- при совершении операций над типами операнды строго должно быть целочисленного типа: byte, int;
 - может содержать вызов функции.
 - двоичные литералы должны заканчиваться суффиксом «b».
 - десятичные литералы должны заканчиваться суффиксом «d» или не иметь суффикса.

Для последующей генерации объектного кода на языке ассемблер все выражения предварительно преобразуются в форму обратной польской записи.

1.14 Конструкции языка

Базовые программные конструкции языка LPA-2025 представлены в таблице 1.8. В неё включены основные синтаксические формы для организации вычислений, управления потоком выполнения и структурирования кода.

Таблица 1.8 – Конструкции языка LPA-2025

Конструкция	Описание	Пример
Функция	<pre><тип данных> function <идентификатор> (<тип_данных> <идентификатор>, ...) {... response <идентификатор литерал> }</pre>	<pre>int function getSum(int a, int b) { return a+b; }</pre>
Условный оператор с выбором из множества возможных значений	<pre>switch (<выражение>) { is <значение_1>: {<инструкции>;} is<значение_2>: {<инструкции>;} any: {<инструкции>;} }</pre>	<pre>switch(val) { is 1: { cout("one"); } is 2: { cout("two"); } is 3: { cout("three"); } any: { cout("other"); } }</pre>
Условный оператор с выбором из двух вариантов	<pre>when(<условие>) {<инструкции языка>} other {<инструкции языка>}</pre>	<pre>when (number > 27) { cout("yes"); } other{ cout("no"); }</pre>

Представленные в таблице конструкции образуют фундамент для построения алгоритмов на языке LPA-2025. Использование функций позволяет структурировать код и повторно использовать логику, оператор when реализует классическое двоичное ветвление, а конструкция switch обеспечивает эффективное многовариантное ветвление. Синтаксис каждой конструкции следует применять в строгом соответствии с приведёнными шаблонами.

1.15 Область видимости идентификаторов

В языке LPA-2025 действуют следующие правила области видимости: все параметры и переменные, объявленные в теле функции, имеют локальную видимость и существуют только в рамках этой функции. Идентификаторы, объявленные на верхнем уровне (вне функций), являются глобальными и доступны из любого места программы. Для формирования дополнительных, вложенных областей видимости применяются блоки кода, ограниченные фигурными скобками { ... }.

1.16 Семантические проверки

Семантические проверки, предусмотренные в языке RIV-2024, представлены в таблице 1.9.

Таблица 1.9 – Семантические проверки

№	Правило
1	Проверка наличия главной функции <code>begin</code> – точки входа в программу
2	Проверка количества точек входа в программу (не должно превышать 1)
3	Запрет совпадения идентификаторов с ключевыми словами
4	Усечение идентификаторов до 9 символов
5	Не допускается совпадения названий идентификаторов с ключевыми словами
6	Все операнды в выражениях и операциях должны иметь типы, допустимые для данного выражения или операции
7	Все идентификаторы должны быть объявлены в той же области видимости перед использованием
8	Наличие логического условия в операторе <code>when</code>
9	Проверка на целочисленное выражение и уникальные значения в <code>switch</code>
10	Возвращаемое значение функции должно совпадать с типом функции

Перечисленные семантические проверки выполняются на этапе компиляции и гарантируют корректность программы с точки зрения правил языка LPA-2025. Соблюдение этих правил обеспечивает предсказуемое поведение программы, предотвращает распространённые ошибки и способствует созданию надежного кода.

1.17 Распределение оперативной памяти на этапе выполнения

Создаваемый компилятором код использует отдельную организацию памяти, состоящую из двух сегментов: сегмента констант, предназначенного для хранения всех литеральных значений, и сегмента данных, в котором размещаются переменные и параметры функций. Для обеспечения локальной области видимости на уровне исходного кода применяется механизм префиксного именования идентификаторов – к именам переменных добавляются специальные префиксы, отражающие их контекст объявления. Это позволяет эмулировать вложенные области видимости на этапе компиляции. Однако при трансляции в низкоуровневый код данная абстракция устраняется: в итоговом ассемблерном представлении все данные размещаются в едином глобальном адресном пространстве, что упрощает управление памятью на этапе выполнения.

1.18 Стандартная библиотека и ее состав

В языке LPA-2025 присутствует стандартная библиотека, которая подключается автоматически при трансляции исходного кода в язык

ассемблера. Содержимое библиотеки и описание функций представлено в таблице 1.10

Таблица 1.10 – Стандартная библиотека языка LPA-2025

Функция	Описание
int comparestr(char* str1, char* str2)	Сравнивает две строки лексикографически. Возвращает: 0 – если строки равны, отрицательное число – если str1 < str2, положительное число – если str1 > str2
int strtoint(char* str)	Преобразует строковое представление числа в целочисленное значение. Возвращает результат преобразования или 0 при ошибке
void writestr(char* str)	Вывод в консоль строковой переменной
void writechar(char ch)	Вывод в консоль символьной переменной
void writeint(int n)	Вывод в консоль целочисленной переменной

Представленный набор встроенных функций обеспечивает выполнение базовых операций ввода-вывода и преобразования данных, которые часто требуются при разработке программ. Эти функции являются частью стандартной библиотеки языка и доступны для использования без дополнительных объявлений. При вызове необходимо строго соблюдать типы передаваемых аргументов, как указано в сигнатурах функций.

1.19 Ввод и вывод данных

Вывод данных осуществляется с помощью ключевого слова «cout». В качестве аргумента могут использоваться числовые и строковые идентификаторы, а также литералы: cout (<идентификатор>|<литерал>).

Ввод данных не предусмотрен.

1.20 Точка входа

Главной функцией и точкой входа в программе является функция с именем «begin». Она должна присутствовать в коде в единственном экземпляре: её объявление является обязательным, а дублирование или переопределение не допускается.

1.21 Препроцессор

На этапе генерации объектного кода подключение стандартных библиотек осуществляется с помощью директивы ассемблера includelib.

1.22 Соглашения о вызовах

Язык программирования LPA-2025 применяет соглашение о вызовах «stdcall». Согласно этому соглашению, все параметры функции передаются через стек в порядке справа налево, а очистку стека от переданных аргументов выполняет сама вызываемая функция перед возвратом управления.

1.23 Объектный код

Язык программирования LPA-2025 транслируется в исходный код на языке ассемблера.

1.24 Классификация сообщений транслятора

В соответствии с префиксами транслятор будет генерировать текст сообщений об ошибке и выводить их в файл протокола. В таблице 1.11 представлены возможные классификации сообщений транслятора.

Таблица 1.11 – Классификация сообщений транслятора

Диапазон	Описание
0 – 3	Системные ошибки
100 – 112	Ошибки параметров
113 – 125	Ошибки лексического анализа
600 – 607	Синтаксические ошибки
700 – 712	Ошибки семантического анализа

Таким образом, каждому типу ошибки соответствует свой диапазон кодов. Эта система позволяет быстро определить категорию проблемы (лексика, синтаксис, семантика) и упрощает работу с протоколом компиляции.

1.25 Контрольный пример

Контрольный пример демонстрирует основные особенности языка LPA-2025. В нём представлены типы данных, использование функций стандартной библиотеки, использование условных операторов, вывод данных, создание функций. Исходный код приведён в приложении А

2. Структура транслятора

2.1. Компоненты транслятора их назначение и принципы взаимодействия

Транслятор — это программа, предназначенная для преобразования исходного кода, написанного на одном языке программирования, в эквивалентный по функциональности код на другом языке программирования. Принцип работы транслятора представлен на рисунке 2.1.

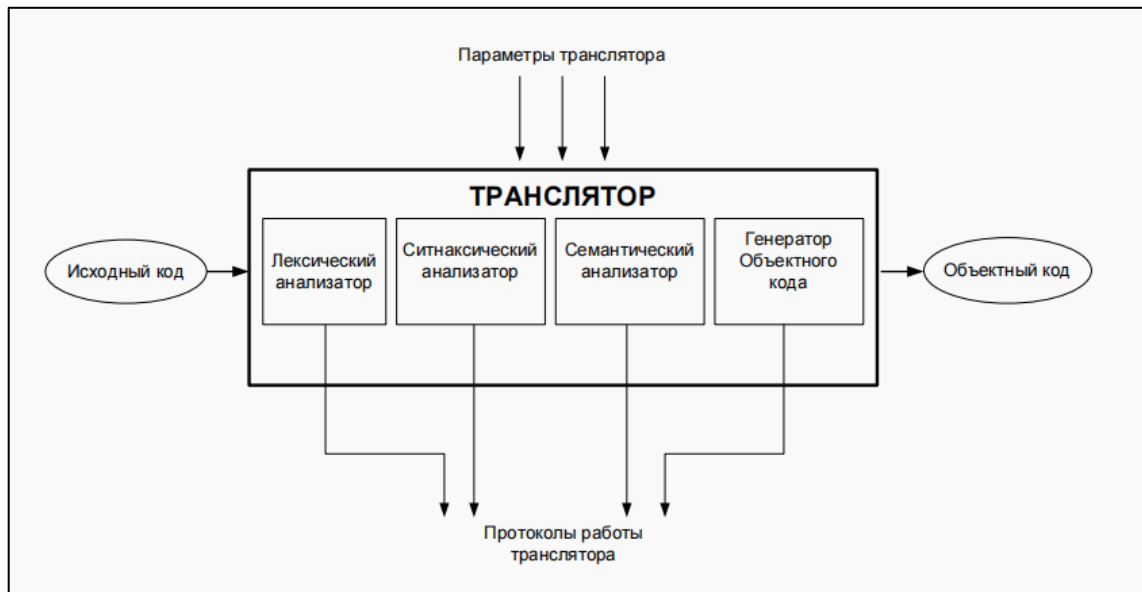


Рисунок 2.1 – Схема работы транслятора

Процесс трансляции состоит из четырёх последовательных этапов: лексического, синтаксического и семантического анализа, а также генерации объектного кода.

Лексический анализ — начальный этап трансляции. Его задача — преобразовать исходный код, очищенный от незначимых символов (пробелов, комментариев), в последовательность лексем. На вход лексер получает файл с исходным кодом, на выходе формируются таблица лексем и таблица идентификаторов.

Синтаксический анализ проверяет, соответствует ли поток лексем формальной грамматике языка. Используя таблицы, полученные на предыдущем этапе, синтаксический анализатор строит дерево разбора, которое отражает иерархическую структуру программы согласно правилам языка.

Семантический анализ выявляет смысловые ошибки, которые нельзя обнаружить на уровне синтаксиса. Он проверяет корректность использования типов данных, областей видимости идентификаторов и соответствие вызовов функций их объявлениям, опираясь на информацию из таблиц лексем и идентификаторов.

Генерация кода – завершающий этап, на котором промежуточное представление программы преобразуется в код на языке ассемблера. Генератор использует таблицы лексем и идентификаторов для создания выходного файла с расширением .asm, содержащего готовый к ассемблированию код.

2.2. Параметры транслятора, управляющие его работой

Работой транслятора можно управлять с помощью входных параметров, передаваемых при его запуске из командной строки или задаваемых в среде разработки. Эти параметры позволяют настраивать процесс компиляции, выбирать выходные форматы и контролировать уровень диагностики. В таблице 2.1 приведены основные управляющие параметры.

Таблица 2.1 – Входные параметры транслятора языка LPA-2025

Входной параметр	Описание	Значение по умолчанию
-in:<путь к in-файлу>	Обязательный. Файл с исходным кодом на языке RIV-2024 с расширением .txt	Не предусмотрено
-log:<путь к log-файлу>	Файл журнала для вывода протоколов работы программы	<имя in-файла>.log
-out:<путь к out-файлу>	Выходной файл с результатом работы транслятора. Содержит исходный код на языке ассемблера	<имя in-файла>.asm

Данные параметры обеспечивают базовое управление процессом трансляции. Их использование позволяет интегрировать транслятор в автоматизированные конвейеры сборки и обеспечивать воспроизводимость результатов за счёт явного указания путей к файлам.

2.3 Протоколы, формируемые транслятором

В процессе работы транслятор генерирует набор выходных файлов (протоколов), каждый из которых содержит специфическую информацию о результатах выполнения отдельных этапов трансляции. Состав формируемых протоколов представлен в таблице 2.2.

Таблица 2.2 – Протоколы, формируемые транслятором LPA-2025

Формируемый протокол	Назначение
1	2
Файл протокола, заданный параметром «-log:»	Основной журнал работы транслятора. Содержит: входные параметры запуска, общую статистику (количество строк, символов), детальный протокол работы синтаксического анализатора, а также все обнаруженные ошибки с указанием номера строки и позиции.

Продолжение таблицы 2.2

1	2
---	---

Выходной файл «IT.txt»	Таблица идентификаторов. Формируется на этапе лексического анализа и содержит информацию обо всех обнаруженных в исходном коде идентификаторах.
Выходной файл «LT.txt»	Таблица лексем. Формируется на этапе лексического анализа и представляет собой последовательность токенов (лексем), выделенных из исходного кода, с указанием их типов и значений.
Выходной файл «LT_modified.txt»	Модифицированная таблица лексем. Содержит последовательность лексем, преобразованную в форму обратной польской записи, подготовленную для последующего этапа генерации кода.
Выходной файл, заданный параметром «-asm: «	Целевой файл с исходным кодом на языке ассемблера. Является конечным результатом работы транслятора, готовым к обработке ассемблером и компоновщиком.

Данные протоколы позволяют отследить трансформацию кода на всех этапах: от лексем и идентификаторов до промежуточного представления и финального ассемблерного кода. Основной лог-файл используется для диагностики и отладки, фиксируя ошибки и ход выполнения.

3. Разработка лексического анализатора

3.1. Структура лексического анализатора

Лексический анализатор выполняет начальный этап трансляции. Его задача — преобразовать последовательность символов исходного кода в упорядоченный поток лексем — базовых элементов языка. Общая схема работы лексического анализатора показана на рисунке 3.1.



Рисунок 3.1 – Схема работы лексического анализатора

На вход анализатора поступает текст программы. В процессе обработки, которая управляется таблицей допустимых символов и списком ключевых слов, формируются две основные структуры данных: таблица лексем и таблица идентификаторов. Подробное описание этих управляющих таблиц и алгоритмов будет приведено в последующих разделах.

3.2. Контроль входных символов

На рисунке 3.2. приведена таблица допустимости символов, используемых для контроля правильности использования исходного текста.

```

#define IN_CODE_TABLE { \
  IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::S, IN::F, IN::F, IN::I, IN::F, IN::F, /*15*/ \
  IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, /*31*/ \
  IN::P, IN::S, IN::S, IN::S, IN::T, IN::S, IN::S, IN::S, IN::S, IN::S, IN::S, IN::S, IN::S, IN::S, IN::S, IN::T, IN::S, /*47*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::S, IN::S, IN::S, IN::S, IN::T, IN::S, /*63*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, /*79*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::S, IN::S, IN::S, IN::T, IN::T, IN::T, /*95*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, /*111*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::S, IN::S, IN::S, IN::S, IN::F, /*127*/ \
  \
  IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, /*143*/ \
  IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, /*159*/ \
  IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, /*175*/ \
  IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, IN::F, /*191*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, /*207*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, /*223*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, /*239*/ \
  IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, IN::T, /*255*/ \
}
  
```

Рисунок 3.2 – Таблица допустимости символов

Таблица допустимости символов основана на кодировке Windows-1251. Индекс элемента соответствует его коду в таблице символов кодировке. Соответствие символов контроля и их назначение представлено в таблице 3.1.

Таблица 3.1 – Соответствие символов и их значений

Символ	Расшифровка	Описание
T	Разрешенный символ	Все используемые символы, разрешенные алфавитом языка
F	Запрещенный символ	Символы, запрещенные для использования
I	Игнорируемый символ	Символы, игнорируемые анализатором
S	Символ-сепаратор	Разграничивают символы алфавита языка между собой
P	Символ-пробел	Разделение токенов

Данная классификация позволяет лексическому анализатору однозначно интерпретировать каждый байт входного потока.

3.3. Удаление избыточных символов

Перед началом лексического анализа исходный код проходит стадию предобработки, целью которой является удаление избыточных пробельных символов. Избыточными считаются последовательные пробелы, а также пробелы в начале или конце строки. Исключение составляют пробелы, находящиеся внутри строковых или символьных литералов.

Принцип удаления избыточных символов:

- Создать строковый буфер для записи обработанного текста.
- Последовательно читать символы из исходного файла.
- Для каждого символа определить его класс по таблице допустимости и выполнить соответствующее действие:
 - a. Допустимый (T): символ добавляется в буфер.
 - b. Запрещенный (F): символ заменяется на маркер «^» и сохраняется в отдельном массиве для дальнейшей диагностики.
 - c. Игнорируемый (I): символ пропускается.
 - d. Пробел (P): пробел добавляется в буфер только в том случае, если он не является избыточным (не в начале/конце строки и не следует за другим пробелом) и не находится внутри незакрытого литерала. В противном случае пробел пропускается.
 - e. Сепаратор (S): символ добавляется в буфер. Если перед сепаратором в буфере находится пробел, он заменяется на данный сепаратор.
- После обработки каждой строки (кроме последней) в буфер добавляется символ переноса строки.
- По завершении обработки всего файла в конец буфера добавляется нуль-символ ('\0'), после чего выделенная память освобождается.

3.4. Перечень ключевых слов

В процессе лексического анализа лексические единицы исходного кода заменяются соответствующими лексемами. Состав ключевых слов языка, распознаваемых анализатором, приведён в таблице 3.2.

Таблица 3.2 – Ключевые слова языка LPA-2025

Фраза LPA-2025	Лексема	Примечание
byte int symb str torf	t	Название типов данных
идентификатор	i	Максимальная длина идентификатора – 9
литералы	l	byte, int, ch или string значение
функция	f	
def	d	Объявление/инициализация переменной
response	r	
cout	p	
begin	m	Точка входа в программу
when	s	Условный оператор, альтернативный оператору if
other	e	Условный оператор, альтернативный оператору else
switch	h	Условный оператор, альтернативный оператору switch
is	c	Условный оператор, альтернативный оператору case
any	a	Условный оператор, альтернативный оператору default
;	;	
,	,	
:	:	
{	{	
}	}	
(	(	
)	)	
+ - * / &	v	Арифметические и бинарные операции
~	~	Побитовая инверсия
[		Префиксный инкремент
]		Префиксный декремент
^		Постфиксный инкремент
_		Постфиксный декремент
=	=	
> <	b	Операции сравнения

Для распознавания лексических конструкций, таких как идентификаторы, литералы и ключевые слова, применяется конечный автомат, заданный набором регулярных выражений. Например, шаблон для идентификатора описывается выражением: `[a-zA-Z_][a-zA-Z0-9_]*`.

Если цепочка успешно распознаётся, она записывается в таблицу лексем. В случае идентификаторов или литералов информация о них также заносится

в таблицу идентификаторов. Пример реализованного конечного автомата для ключевого слова `response` языка LPA-2025 представлен в листинге 3.1.

```
#define FST_RESPONSE FST::FST _response(str, \
    9, \
    FST::NODE(1, FST::RELATION('r', 1)), \
    FST::NODE(1, FST::RELATION('e', 2)), \
    FST::NODE(1, FST::RELATION('s', 3)), \
    FST::NODE(1, FST::RELATION('p', 4)), \
    FST::NODE(1, FST::RELATION('o', 5)), \
    FST::NODE(1, FST::RELATION('n', 6)), \
    FST::NODE(1, FST::RELATION('s', 7)), \
    FST::NODE(1, FST::RELATION('e', 8)), \
    FST::NODE() \
);
```

Листинг 3.1 – Пример реализации графа конечного автомата LPA-2025

Приведённый граф демонстрирует базовый принцип распознавания ключевых слов: каждому символу соответствует строго определённый переход между состояниями автомата. Такой механизм обеспечивает точное и безошибочное выделение ключевых слов из потока символов. Аналогичные автоматы строятся для всех остальных ключевых слов и операторов из таблицы 3.2, что в совокупности образует ядро лексического анализатора языка LPA-2025.

3.5. Основные структуры данных

Для хранения результатов лексического анализа используются две ключевые структуры данных: таблица лексем (LexTable) и таблица идентификаторов (IdTable).

Таблица лексем (LexTable) содержит упорядоченный список всех лексем, сгруппированных по строкам исходного кода. Её общая структура описана в таблице 3.3.

Таблица 3.3 – Описание структуры таблицы лексем

Поле	Назначение
<code>maxsize</code>	максимальное количество элементов в таблице
<code>size</code>	текущее количество элементов в таблице
<code>table</code>	массив элементов типа <code>LT::Entry</code> , являющихся хранилищем для лексем

Каждый элемент массива типа `Entry`, находящего в пространстве имен `LT` представляет отдельную лексему. В ней содержится информация о соответствующей лексем и ее номере в строке исходного кода. В таблице 3.4 представлено описание структуры `LT::Entry`.

Таблица 3.4 – Описание структуры Entry для LexTable

Поле	Назначение
lexema[]	лексема
size	номера строка в исходном коде
idxTI	индекс в таблице идентификаторов. По умолчанию устанавливается значение -1.

Таблица идентификаторов (IdTable) имеет аналогичную структуру данных, как и у таблицы лексем. Отличием является массив Entry, находящийся уже в пространстве имен IT, который имеет другую реализацию для IdTable. Описание структуры IT::Entry представлено в таблице 3.5.

Таблица 3.5 – Описание структуры Entry для IdTable

Поле	Назначение
idxfirstLE	индекс данного идентификатора в таблице лексем
line	номер текущей линии
id	имя идентификатора
iddatatype	тип данных языка LPA-2025
idtype	Тип идентификатора. Принимает значения переменная, функция или параметр
scope	Область видимости идентификатора
value	Объединение полей, используемых для хранения значений литералов типа данных языка LPA-2025

Программная реализация структур LexTable и IdTable представлена в приложении Б.

3.6. Структура и перечень сообщений лексического анализатора

Диагностическое сообщение содержит уникальный код, текстовое описание проблемы, а также координаты её расположения в исходном коде – номер строки и позицию.

Полный список возможных ошибок, выявляемых на этапе лексического анализа, приведён в таблице 3.6.

Таблица 3.6 – Перечень сообщений лексического анализатора LPA-2025

Номер	Сообщение
1	2
113	Недопустимый идентификатор
114	Дублирование идентификатора
115	Идентификатор не определён
116	Некорректное использование индекса
117	Недопустимый размер таблицы идентификаторов
118	Превышен размер таблицы идентификаторов
119	Превышена длина лексемы
120	Функция не может быть типа symb
121	Использование запрещенного символа

Продолжение таблицы 3.6

1	2
122	Кавычки не были закрыты
123	Литералы типа <code>ymb</code> должен содержать только 1 символ
124	Превышен максимальный размер для литерала типа <code>str(255)</code>
125	Превышена максимальная длина идентификатора

Каждое сообщение из данного списка соответствует конкретному нарушению правил языка, определённых на этапе лексического анализа. Указание точного местоположения (строка, позиция) позволяет программисту быстро найти и исправить ошибку в исходном коде. Такая система диагностики обеспечивает эффективную отладку на ранней стадии процесса трансляции.

3.7. Принцип обработки ошибок

Обнаружение любой ошибки на этапе лексического анализа приводит к немедленному прекращению работы анализатора. При этом сформированное диагностическое сообщение, содержащее код ошибки, её описание и точные координаты в исходном коде, выводится в консоль для оперативного информирования пользователя и параллельно записывается в файл журнала (лог-файл) для последующего анализа. Данный подход, предполагающий остановку после первой же найденной ошибки, означает, что за один сеанс работы лексический анализатор может выявить не более одной лексической ошибки. Это требует от разработчика последовательного исправления ошибок и повторного запуска транслятора.

3.8. Параметры лексического анализатора

Входные параметры, используемые в работе лексического анализатора, их назначение и принцип применения описаны в таблице 3.7.

Таблица 3.7 – Входные параметры лексического анализатора LPA-2025

Параметр	Назначение	Принцип применения
Исходный код на языке LPA-2025	Основной входной параметр, представляющий собой текст программы, подлежащий анализ	Построчно анализируется и разбивается на лексемы
Ключевые слова	Задаёт список зарезервированных слов, операторов и специальных символов, используемых в языке; нельзя использовать как идентификаторы.	Лексический анализатор использует ключевые слово для сравнения с обнаруженными токенами
Таблица допустимости входных символов	Определяет, какие символы считаются допустимыми в исходном коде.	Лексический анализатор различает символы, определённые в наборе, и игнорирует остальные

Таким образом, для корректной работы лексический анализатор использует три обязательных компонента: исходный код как объект анализа, список ключевых слов для распознавания языковых конструкций и таблицу допустимости для классификации символов. Их согласованное применение обеспечивает основу для выделения лексем и первичной проверки корректности программы.

3.9. Алгоритм лексического анализа

Алгоритм работы лексического анализатора LPA-2025

- Чтение и буферизация символов: Исходный код обрабатывается посимвольно. Каждый символ записывается в строковый буфер до тех пор, пока не будет обнаружен символ-разделитель (пробел, оператор, скобка и т.д.).
- Распознавание лексемы: Содержимое буфера передаётся на обработку конечному автомату. Если автомат успешно распознаёт лексему, определяется её тип. Для ключевых слов и операторов лексема записывается в таблицу лексем, и процесс возвращается к шагу 1. Если лексема является идентификатором, литералом или точкой входа (begin), алгоритм переходит к следующему шагу.
- Определение типа идентификатора: если лексема является идентификатором, её категория (переменная, параметр, функция) определяется на основе контекста – предыдущих лексем, таких как ключевые слова `def` или объявление функции.
- Обработка объявления идентификатора: если идентификатор объявляется впервые, в таблице идентификаторов выполняется поиск по имени для проверки на дублирование. Перед поиском фиксируются его тип данных и область видимости (из текущего контекста в стеке). Если идентификатор с таким именем уже существует, анализ завершается с ошибкой. В противном случае в таблицу идентификаторов добавляется новая запись, а в таблицу лексем записывается лексема с ссылкой на эту запись. Затем алгоритм возвращается к шагу 1.
- Обработка использования идентификатора: если идентификатор используется (не объявляется), выполняется его поиск в таблице идентификаторов. Если идентификатор не найден, анализ завершается с ошибкой. Если найден, лексема записывается в таблицу лексем с ссылкой на существующую запись. Затем алгоритм возвращается к шагу 1.
- Обработка литерала: определяется тип и значение литерала. В таблице идентификаторов выполняется поиск идентичной записи литерала. Если литерал уже существует, новая запись не создаётся. Если не существует – литерал добавляется в таблицу идентификаторов. В таблицу лексем записывается лексема с ссылкой на соответствующую запись. Затем алгоритм возвращается к шагу 1.

– Обработка функции: если идентификатор является функцией, он добавляется в таблицу идентификаторов с указанием типа возвращаемого значения. Все параметры, перечисленные в круглых скобках, также добавляются как записи в таблицу идентификаторов с типом «параметр». Для отслеживания области видимости в стек помещается метка функции, которая сохраняется до конца её объявления. Затем алгоритм возвращается к шагу 1.

– Проверка завершения: если исходный код не обработан полностью, алгоритм возвращается к шагу 1. В противном случае работа лексического анализатора завершается.

3.10. Контрольный пример

Результат работы лексического анализатора, полученного при выполнении контрольного примера, представлен в приложении В.

4. Разработка синтаксического анализатора

4.1. Структура синтаксического анализатора

Синтаксический анализатор – это вторая фаза компилятора, которая проверяет синтаксис программы согласно формальной грамматике языка LPA-2025. Он получает на вход таблицы лексем и идентификаторов, а в результате формирует дерево разбора, отражающее структуру программы.

На рисунке 4.1 будет представлена схема работы синтаксического транслятора.

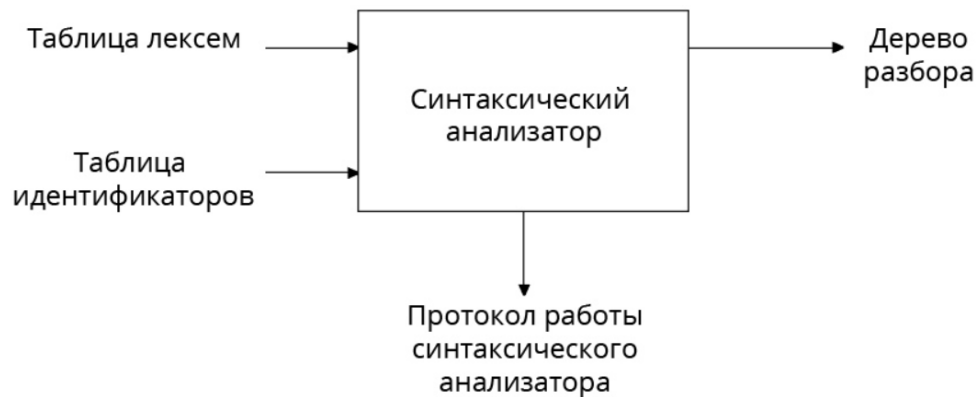


Рисунок 4.1 – Схема работы синтаксического анализатора

Анализатор проверяет лексемы по правилам грамматики, выявляя синтаксические ошибки и формируя дерево разбора для следующих этапов компиляции.

4.2. Контекстно-свободная грамматика, описывающая синтаксис языка

В синтаксическом анализаторе транслятора языка RIV-2024 используется контекстно-свободная грамматика $G=\langle T, N, P, S \rangle$, где:

T – множество терминальных символов (набор лексем, перечисленных в таблице 3.2);

N – множество нетерминальных символов (первый столбец таблицы 4.1);

P – множество правил языка (второй столбец таблицы 4.1);

S – начальный символ грамматики, являющийся нетерминалом.

Контекстно-свободная грамматика $G=\langle T, N, P, S \rangle$ имеет нормальную форму Грейбах, если она не леворекурсивная (не содержит леворекурсивных правил), а правила P имеют вид:

$A \rightarrow \alpha a A$, где $a \in T$, $\alpha \in (N \cup T)^*$;

$S \rightarrow \lambda S$, где $S \in N$ – начальный символ, и если такое правило существует, то нетерминал не должен встречаться в правой части правил.

Полный перечень синтаксических правил в форме Грейбах перечислены в таблице 4.1.

Таблица 4.1 – Синтаксические правила LPA-2025

Нетерминал	Цепочка	Описание
S	$m\{N\}$	Стартовый символ порождает всю структуру исходного кода.
	$tft(N)\{N\};S$	
	$m\{N\};s$	
N	$dti;$	Операторы программы
	$dti;N$	
	dtN	
	$dti=E;N$	
	$i=E;$	
	$i=E;N$	
	$rE;$	
	$rE;N$	
	$pE;$	
	$pE;N$	
	$s(N)\{N\}$	
	$s(N)\{N\}N$	
	$s(N)\{N\}N\ e\{N\}N$	
E	i	Выражения
	l	
	iM	
	lM	
	(E)	
	$i(W)$	
	$i()$	
	$(E)M$	
	$i(W)M$	
	ivi	
	$\sim E$	
	$E \vee E$	
	$(E) \vee E$	
	$R(W)$	
	$M(W)$	
W	i	Подвыражения
	l	
	i, W	
	l, W	
F	ti	Параметры функции
	ti, F	
M	vE	Выражение (арифметические операции и операции сравнения)
	vEM	
	$(E)v$	
	$(E)v(E)$	
B	ibi	Операции сравнения
	ibl	
	lbi	
	lbl	

Представленная грамматика задаёт все синтаксически корректные конструкции языка LPA-2025. Она используется синтаксическим анализатором как набор правил для проверки структуры программы и построения дерева разбора.

4.3. Построение конечного магазинного автомата

Конечный автомат с магазинной памятью представляет собой семерку $M = \langle Q, V, Z, \delta, q_0, z_0, F \rangle$. Подробное описание компонентов магазинного автомата представлено в таблице 4.2.

Таблица 4.2 – Описание компонентов магазинного автомата

Компонента	Определение	Описание
Q	Множество состояний автомата	Состояние автомата представляет из себя структуру, содержащую позицию на входной ленте, номера текущего правила и цепочки и стек автомата
V	Алфавит входных символов	Алфавит представляет из себя множества терминальных и нетерминальных символов, описание которых содержится в таблица 3.2 и 4.1 соответственно
Z	Алфавит специальных магазинных символов	Алфавит магазинных символов содержит стартовый символ и маркер дна стека (представляет из себя символ \$)
δ	Функция переходов автомата	Функция представляет из себя множество правил грамматики, описанных в таблице 4.1
q_0	Начальное состояние автомата	Состояние, которое приобретает автомат в начале своей работы. Представляется в виде стартового правила грамматики
z_0	Начальное состояние магазина автомата	Символ маркера дна стека \$
F	Множество конечных состояний	Конечные состояние заставляют автомат прекратить свою работу. Конечным состоянием является пустой магазин автомата и совпадение позиции на входной ленте автомата с размером ленты

Данные компоненты формально определяют магазинный автомат для синтаксического анализатора LPA-2025. Его работа заключается в проверке соответствия потока лексем правилам грамматики (таблица 4.1) с использованием стека для отслеживания структуры программы. Корректный

разбор завершается при пустом стеке и полном прочтении входной последовательности.

4.4. Основные структуры данных

В таблице 4.3 представлено описание структур, характеризующих контекстно-свободную грамматику.

Таблица 4.3 – Описание структуры данных синтаксического анализатора

Структура	Описание
1	2
Greibach	Все символы (алфавит) грамматики представляются в формате GRBALPHABET (short). Причем терминалы – положительные значения, нетерминалы – отрицательные значения. Структура включает: – множество правил: переменная rules (типа структура Rule); – количество правил: переменная size (short); – стартовый символ грамматики: startN (GRBALPHABET); – служебный символ (дно стека и последняя лексема таблице лексем): stbottomT (GRBALPHABET); – два конструктора; – методы getRule: позволяет получить номер правила или -1 (к точке возврата) и правило (второй параметр типа Rule) по левому символу правила (первый параметр); позволяет получить правило (возвращает к точке вызова параметр типа Rule) по его номеру.
Rule	Структура: представление одного правила, имеющего вид: $A \rightarrow xxx yuu$ Структура включает: – нетерминал – левый символ правила: nn (GRBALPHABET); – идентификатор ошибки, связанной с правилом: iderror (int) – код ошибки в подсистеме Eerror; – количество цепочек в правой стороне правила: size (short); – цепочки-правые стороны правила: chains (типа Rule::Chain); – два конструктора; – метод getCRule: позволяет получить правило в виде строки вида N->цепочка (в символьном ASCII-виде, для отображения); – метод getNextChain: позволяет найти следующую за заданным – номером (3й параметр j типа short) цепочку (параметр pchain типа Rule::Chain) и ее номер (к точке возврата типа short).
Rule::Chain	Структура: представление цепочки – правой стороны правила. Структура включает: – размер цепочки: size (short) в символах; – цепочка: nt (GRBALPHABET); – два конструктора; – метод getCChain: позволяет получить строку-цепочку в – символьном виде для отображения;

Продолжение таблицы 4.3

1	2
Rule::Chain	– методы T и N: преобразовывают ASCII-символы в GRBALPHABET-символы (терминалы и нетерминалы); – методы isT и isN: проверяют является GRBALPHABET-символ терминалом или нетерминалом; – метод alphabet_to_char: преобразует заданный (параметр) GRBALPHABET-символ в ASCII-символ.

Реализация структур данных семантического анализа на языке C++ представлена в приложении Г.

4.5. Описание алгоритма синтаксического разбора

Алгоритм синтаксического разбора (магазинного автомата) реализуется следующим образом.

- Инициализируется стек, в который помещается стартовый символ грамматики.
- На основе таблицы лексем формируется входная последовательность (лента) для анализа.
- Если на вершине стека находится нетерминальный символ, он заменяется на одну из допустимых правых частей правил грамматики, а полученная цепочка символов (в обратном порядке) помещается в стек.
- Если на вершине стека находится терминальный символ, он сравнивается с текущим символом входной ленты. При совпадении оба продвигаются; при несовпадении выполняется откат к предыдущему сохранённому состоянию и выбирается альтернативное правило.
- Анализ повторяет шаги 3–4, пока не будет достигнуто дно стека.
- Разбор считается успешным, если в момент опустошения стека входная лента также полностью прочитана. В противном случае фиксируется синтаксическая ошибка.

4.6. Структура и перечень сообщений синтаксического анализатора

Синтаксический анализатор в процессе работы может обнаруживать различные виды ошибок, такие как несоответствие грамматическим правилам, отсутствие или неверное расположение ключевых элементов программы. Полный перечень возможных ошибок, которые способен выявить анализатор, представлен в таблице 4.4.

Таблица 4.4 – Перечень сообщений синтаксического анализатора

Код	Сообщение
1	2
600	Неверная структура программы
601	Ошибочный оператор
602	Ошибка в выражении

Продолжение таблицы 4.4

1	2
603	Ошибка в подвыражении
604	Ошибка в параметрах функции
605	Ошибка в арифметических операторах
606	Ошибка в операторах сравнения
607	Синтаксический анализ завершён досрочно

Каждая ошибка соответствует конкретному нарушению синтаксиса языка. При обнаружении любой из них трансляция прерывается, а диагностическое сообщение записывается в протокол.

4.7. Параметры синтаксического анализатора и режимы его работы

Синтаксическому анализатору на вход поступают таблицы, сформированные на предыдущем этапе: таблица лексем и таблица идентификаторов. Для управления процессом разбора используется формальное описание грамматики языка в нормальной форме Грейбах. Результатом работы синтаксического анализатора являются дерево разбора программы и подробный протокол работы магазинного автомата, который записывается в журнал.

4.8. Принцип обработки ошибок

Принцип обработки ошибок синтаксическим анализатором:

- Анализатор проверяет соответствие последовательности лексем правилам грамматики, используя алгоритм магазинного автомата.
- Если подходящее правило не найдено, формируется классифицированное диагностическое сообщение с привязкой к строке и позиции в исходном коде.
- Обнаруженная ошибка не останавливает разбор. Она сохраняется в общий список, после чего анализатор выполняет восстановление и продолжает работу, выявляя дальнейшие ошибки.
- По завершении анализа все накопленные ошибки сортируются по расположению в коде и выводятся в протокол, формируя полный отчёт для исправления.

4.9. Контрольный пример

Результат работы протокола синтаксического разбора и дерево разбора представлены в приложение Д.

5. Разработка семантического анализатора

5.1. Структура семантического анализатора

Семантический анализ представляет собой третью, смысловую фазу работы транслятора. Его задача – проверить исходную программу на соответствие семантическим (смысловым) правилам языка, которые невозможно описать одной только формальной грамматикой.

Для работы анализатор использует результаты предыдущих этапов: синтаксическое дерево, отражающее структуру программы, и заполненную таблицу идентификаторов. На их основе он выполняет проверки типов данных, областей видимости, соответствия параметров функций и других контекстно-зависимых правил.

Все обнаруженные семантические ошибки и предупреждения записываются в файл протокола транслятора. Общая структура и взаимосвязь компонентов семантического анализатора представлена на рисунке 5.1.

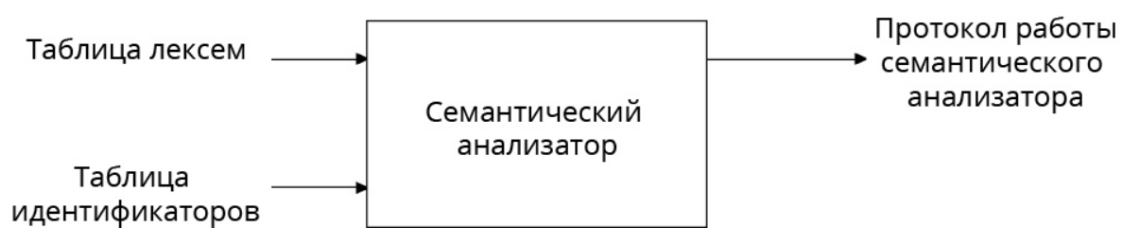


Рисунок 5.1 – Схема работы семантического анализатора

Как показано на схеме, анализатор проверяет узлы дерева, используя информацию из таблиц, и формирует протокол с найденными ошибками.

5.2. Функции семантического анализатора

Полный перечень семантических проверок с указанием их фаз приведён ниже в таблице 5.1.

Таблица 5.1 – Перечень семантических проверок

№	Правило	Фаза
1	2	3
1	Наличие begin	Синтаксический
2	Дублирование begin	Лексический
3	Идентификаторы усекаются до 9 символов	Лексический
4	Все идентификаторы должны быть объявлены в той же области видимости перед использованием	Лексический
5	Не допускается совпадения названий идентификаторов с ключевыми словами	Лексический
6	Соответствие типов формальных и фактических параметров при вызове функций	Семантический

Продолжение таблицы 5.1

1	2	3
7	Соответствие типов формальных и фактических параметров при вызове операторов	Семантический
8	Проверка целочисленного типа выражения в switch	Семантический
9	Уникальность константных значений в ветках is	Семантический
10	Возвращаемое значение функции должно совпадать с типом функции	Семантический

Семантический анализатор имеет набор функций, реализующий проверку некоторых семантических правил. На вход подается таблица лексем и таблица идентификаторов, и позиция, с которой начинается проверка. Перечень функций с их описанием представлен в таблице 5.2

Таблица 5.2 – Перечень функции семантического анализатора

Функция	Описание
functions(lex);	Обработка функций, их параметров и возвращаемого значения
literals(lex);	Проверка типа литералов
operands(lex);	Проверка типа операндов

Семантический анализ выполняется путём последовательного вызова специализированных функций при обходе дерева. Каждая функция проверяет определённый класс конструкций на соответствие правилам языка.

5.3. Структура анализатора

Перечень сообщений, формируемых семантическим анализатором в ходе своей работы, представлен в таблице 5.3.

Таблица 5.3 – Перечень функции семантического анализатора

Код	Сообщение
700	Тип возвращаемого значения функции не соответствует возвращаемому значению
701	Все переменные должны инициализироваться
702	Не совпадают типы параметров функции при вызове и объявлении
703	Типы данных, используемые в выражении, не совпадают
704	Попытка вызвать операцию, неподдерживаемую типом данных
705	Превышение максимального количества параметров функции
706	Невозможно использовать идентификатор функции кроме как для её вызова
707	Попытка присвоить идентификатору значение функции, которое отлично от типа данных идентификатора
708	Параметры функции стандартной библиотеки ошибочны
709	cout обязательно должно принимать какое-либо значение
710	Тип is не соответствует типу switch
711	Дублирование значения в is
712	Параметр switch должен быть целочисленного типа

Все сообщения соответствуют семантическим ошибкам.

5.4. Принцип обработки ошибок

Семантический анализатор работает в режиме немедленного прерывания при первой ошибке. Это означает, что при обнаружении любого нарушения семантических правил процесс семантического анализа, а вместе с ним и вся трансляция, немедленно прекращаются.

Перед остановкой сформированное диагностическое сообщение с кодом ошибки, описанием и точной локализацией в исходном коде записывается в файл протокола, заданный соответствующим входным параметром транслятора.

Данный подход, предполагающий остановку после первой же найденной семантической ошибки, означает, что за один запуск семантического анализа (и транслятора в целом) может быть выявлена только одна ошибка этого класса.

5.5. Контрольный пример

Перечень контрольных примеров, демонстрирующих работу семантического анализатора, представлен в таблице 5.4.

Таблица 5.4 – Перечень контрольных примеров

Код	Сообщение
<pre>byte task getInv(byte as) { response ~as; } begin { getInv = 3; }</pre>	Ошибка 706: Невозможно использовать идентификатор функции кроме как для её вызова Строка: 7, позиция: 0
<pre>begin { def byte sw = 1; switch(sw){ is 1:{cout("one1");} is 1:{cout("one2");} any:{cout("any");} } }</pre>	Ошибка 711: Дублирование значения в is Строка: 5, позиция: 3

Приведённые примеры наглядно демонстрируют типичные семантические ошибки, которые обнаруживает анализатор.

6. Вычисление выражений

6.1. Выражения, допускаемые языком

В языке LPA-2025 допускаются вычисления выражений целочисленного типа данных с поддержкой вызова функций внутри выражений. Приоритет операций представлен на таблице 6.1.

Таблица 6.1 – Приоритеты операций

Операция	Приоритет
()	0
~	4
^ (постфиксный инкремент)	4
_ (постфиксный декремент)	4
[(префиксный инкремент)	4
] (префиксный декремент)	4
* /	3
+ -	2
< >	1
&	1
, :	1

Примечание: операции с приоритетом «0» имеют высший приоритет. Следующие ниже по приоритету с «4», и далее по убыванию.

6.2. Польская запись и принцип ее построения

Все выражения языка LPA-2025 преобразовываются к обратной польской записи.

Польская запись – это альтернативный способ записи арифметических выражений, преимущество которого состоит в отсутствии скобок.

Обратная польская нотация – это форма записи математических выражений, в которой операторы расположены после своих операндов.

Принципы построения польской записи:

- элементы исходного выражения читаются слева направо;
- механизм поочередно извлекает лексему из выражения;
- если текущий символ – идентификатор или литерал, то записываем его в результирующую строку. При этом, если идентификатор является функцией, записывается специальный токен «@» и количество передаваемых параметров;
- открывающаяся скобка автоматически заносится в стек операций;
- закрывающаяся скобка выталкивает все операции из стека в финальную;
- строку и самоуничтожается вместе с открывающейся скобкой;
- операция выталкивает все операции с большим приоритетом в результирующую строку;
- когда встречается символ конца строки кода («;») – все операции выталкиваются из стека в финальную строку;

– элементы таблицы лексем, которые остались свободными, заполняются символом-заполнителем «#».

6.3. Программная реализация обработки выражений

Реализация алгоритма преобразования выражений к польской записи представлена в приложении Е.

6.4. Контрольный пример

Преобразование выражений в формат польской записи необходимо для построения более простых алгоритмов при последующей обработке таблицы лексем.

Пример преобразования выражения к польской записи представлен в таблице 6.2.

Таблица 6.2 – Преобразование выражений к ПОЛИЗ

Исходное выражение	Стек	Результирующая строка
$(i - (1 * i)) - 1$		
$i - (1 * i) - 1$	(	
$-(1 * i) - 1$	(	i
$(1 * i) - 1$	-(	i
$1 * i) - 1$	-(	i
$* i) - 1$	-(	il
$i) - 1$	*(-	il
$) - 1$	*(-	ili
$) - 1$	-(	ili*
$- 1$		ili*-
1	-	ili*-
		ili*-1-

Таким образом, использование стека позволяет корректно упорядочить операторы и операнды, формируя выражение в обратной польской записи. Это устраняет необходимость в скобках и подготавливает выражение для простого последовательного вычисления.

7. Генерация кода

7.1. Структура генератора кода

В качестве целевого языка трансляции выбран Microsoft Macro Assembler (MASM). Генерация объектного кода основана на принципе соответствия: каждая конструкция языка LPA-2025 (объявление переменной, арифметическое выражение, вызов функции, условный оператор) имеет заранее заданный, шаблонный эквивалент в виде последовательности команд ассемблера.

Функции исходного языка транслируются в ассемблерные процедуры. Передача параметров и возврат значений организуются через соглашение о вызовах `stdcall`, которое использует стек и регистры процессора. Условные операторы, такие как “when” и “switch”, реализуются через систему меток и команд условного (“je”, “jne”, “jg”) и безусловного (“jmp”) перехода, что обеспечивает корректное ветвление логики программы. Таким образом, генератор кода преобразует семантически проверенное промежуточное представление в низкоуровневые инструкции, готовые для ассемблирования и компоновки в исполняемый модуль.

В случае возникновения ошибок генерация кода не будет осуществляться.

Структура генератора кода LPA-2025 представлена на рисунке 7.1.

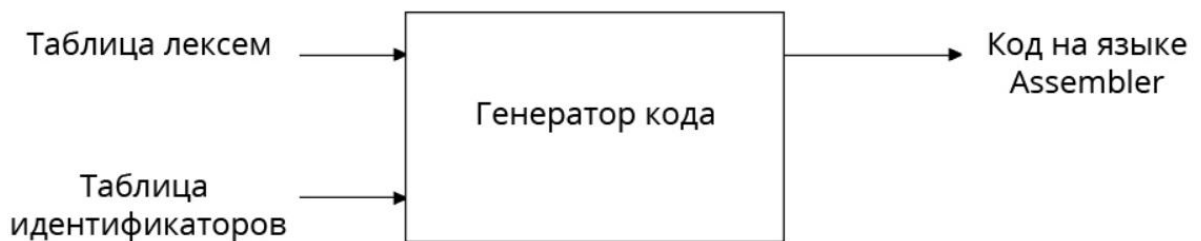


Рисунок 7.1 – Структура генератора кода

Как показано на схеме, генератор кода использует таблицы лексем и идентификаторов в качестве входных данных для формирования последовательности ассемблерных команд на языке MASM, следуя установленным шаблонам трансляции.

7.2. Представление типов данных в оперативной памяти

Элементы таблицы идентификаторов размещаются в соответствующих сегментах памяти языка ассемблер: переменные и параметры – в сегменте `.data`, а литералы (константы) – в сегменте `.const`. Соответствие между типами данных языка LPA-2025 и их ассемблерными представлениями приведено в таблице 7.1.

Таблица 7.1 – Соответствия типов идентификаторов языка и ассемблера

Тип идентификатора на языке LPA-2025	Тип идентификатора на языке ассемблера	Пояснение
byte	sbyte	Хранит целочисленный тип данных размером один байт
symb	byte	Хранит символьный тип данных размером один байт
torf	byte	Хранит логическое значение true, false размером 1 байт
int	dword	Хранит целочисленный тип данных размером 4 байта
str	db	Каждый символ строки типа str хранится в поле размером 1 байт

Таким образом, каждому типу данных LPA-2025 ставится в соответствие определённый директив ассемблера, задающий размер и способ хранения переменной или константы в памяти. Это отображение является основой для корректного выделения памяти и формирования операций при генерации машинного кода.

7.3. Статическая библиотека

В языке LPA-2025 предусмотрена статическая библиотека. Статическая библиотека содержит функции, написанные на языке C++. Объявление функций статической библиотеки генерируется автоматически в коде ассемблера. Перечень функций стандартной библиотеки представлен в таблице 7.2

Таблица 7.2 – Функции статических библиотек

Функция	Назначение
int atoi (char* str)	Преобразование строки в число
int strcmp(char* str1, char* str2)	Сравнение двух строк
void writestr	Вывод в консоль строковой переменной
void writechar	Вывод в консоль символьной переменной
void writeint	Вывод в консоль целочисленной переменной

Таким образом, статическая библиотека расширяет возможности LPA-2025, предоставляя готовые реализации часто требуемых операций.

7.4. Особенности алгоритма генерации кода

Генератор кода языка LPA-2025 формирует ассемблерный текст на основе информации из таблиц лексем и идентификаторов. В основе процесса лежит использование набора заранее подготовленных шаблонных функций и макросов, которые обеспечивают структурированность, оптимизацию и надёжность выходного кода. Эти шаблоны реализуют базовые операции языка: вывод данных, арифметические и логические вычисления, а также механизмы

условного ветвления. Их подключение через вызовы в объектный код упрощает саму генерацию и минимизирует потенциальные ошибки. Общая схема работы генератора кода представлена на рисунке 7.2.

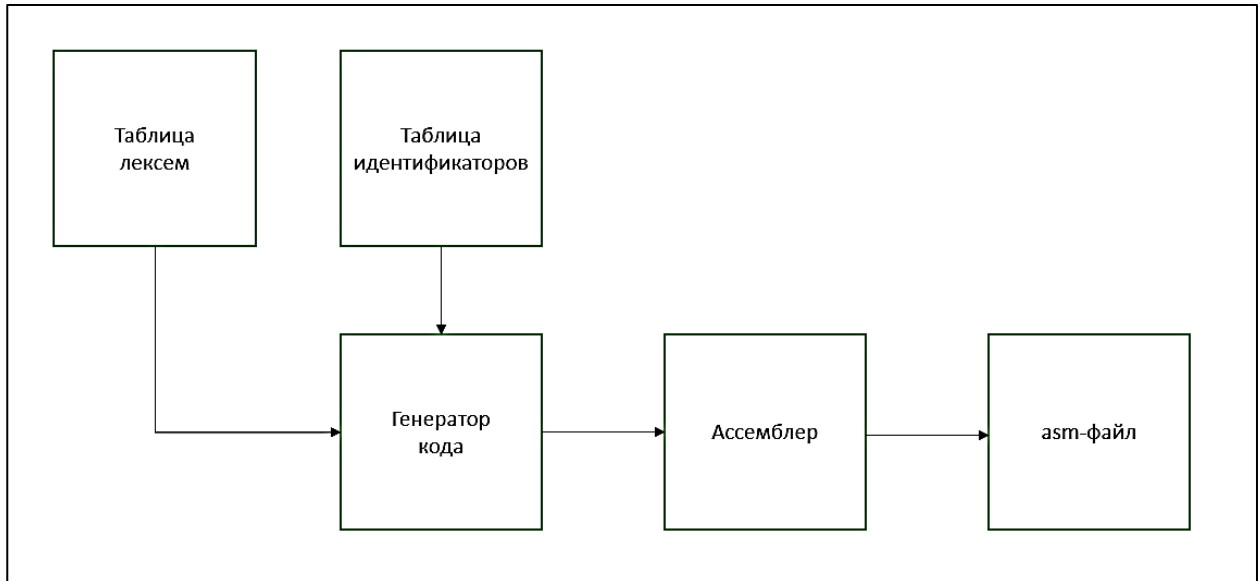


Рисунок 7.2 – Структура генератора кода

Этот подход позволяет эффективно транслировать высокоуровневые операции в низкоуровневые команды, сохраняя семантику исходной программы.

7.5. Входные параметры, управляющие генерацией кода

На вход генератору кода поступают таблицы лексем и идентификаторов исходного кода программы на языке LPA-2025. Результаты работы генератора кода выводятся в файл с расширением .asm.

7.6. Контрольный пример

Результат генерации ассемблерного кода на основе контрольного примера из приложения А приведен в приложении Ж.

8. Тестирование транслятора

8.1. Общие положения

Тесты используются для проверки транслятора на работоспособность и для выявления недочетов и ошибок, а также последующего их исправления.

При возникновении ошибки транслятор немедленно завершает свою работу и выводит сообщение об ошибке с соответствующим ей кодом в консоль и в файл протокола .log.

8.2. Результаты тестирования

В таблице 8.1 приведены ошибки, генерируемые в процессе считывания входного файла, а также в процессе лексического, синтаксического и семантического анализов.

Таблица 8.1 – Результаты тестирования транслятора

Исходный код	Диагностическое сообщение	Этап
1	2	3
<pre>byte task test(byte a) { response a; } byte task test(byte b) { response b; } begin { ... }</pre>	Ошибка 114: Дублирование идентификатора Строка: 5, позиция: 10	Лексический
<pre>begin { def byte f = 3; def byte f = 4; }</pre>	Ошибка 114: Дублирование идентификатора Строка: 3, позиция: 14	Лексический
<pre>begin { cout(f); }</pre>	Ошибка 115: Идентификатор не определён Строка: 2, позиция: 11	Лексический
<pre>begin { ... } begin { ... }</pre>	Ошибка 116: Дублирование begin	Лексический
<pre>symb task test(symb a) { response a; } begin { ... }</pre>	Ошибка 120: Функция не может быть типа symb Строка: 1, позиция: 17	Лексический

Продолжение таблицы 8.1

1	2	3
def byte flag = 2; cout(flag);	600: строка 1, Неверная структура программы	Синтаксический
begin { def byte sw = 1; switch(sw){ is 1:{cout("one1");} is 1:{cout("one2");} any:{cout("any");} } }	Ошибка 711: Дублирование значения в is Строка: 5, позиция: 3	Семантический

Приведённые примеры демонстрируют, как транслятор идентифицирует и классифицирует ошибки на разных этапах обработки. Тестирование подтверждает, что система диагностики корректно работает для различных типов нарушений: от дублирования идентификаторов и неверной структуры программы до семантических противоречий в операторах.

Заключение

В ходе выполнения курсового проекта была достигнута основная цель – разработан собственный язык программирования LPA-2025 и компилятор для его обработки. Выполненные задачи позволили изучить основные аспекты создания языка программирования и транслятора, включая определение алфавита, синтаксиса и семантики языка, а также реализацию основных этапов трансляции.

В рамках проекта были реализованы следующие ключевые этапы:

- разработана спецификация языка LPA-2025;
- реализованы лексический, синтаксический и семантический анализаторы, обеспечивающие обработку исходного текста программы.
- разработан процесс генерации объектного кода, демонстрирующий функциональные возможности языка.
- создан и протестирован контрольный пример программы, написанной на языке LPA-2025, для проверки работоспособности всех компонентов транслятора.
- Окончательная версия языка LPA-2025 включает:
 - 5 типов данных;
 - поддержка оператора вывода строки;
 - возможность вызова функций стандартной библиотеки;
 - наличие 3 унарных, 4 арифметических и 2 логических операторов для вычисления выражений;
 - поддержка функций, оператора условия с выбором из множества возможных вариантов и оператор условия с выбором из двух значений;
 - структурированная и классифицированная система для обработки ошибок пользователя.

Проделанная работа позволила получить необходимое представление о структурах и процессах, использующихся при построении трансляторов, а также основные различия и преимущества тех или иных средств трансляции.

Список используемых источников

1. Курс лекций по КПО Наркевич А. С.
2. Ахо, А. Компиляторы: принципы, технологии и инструменты / А. Ахо, Р. Сети, Дж. Ульман. – М.: Вильямс, 2003. – 768 с.
3. Прата, С. Язык программирования C++. Лекции и упражнения / С. Прата. – М., 2006 – 1104 с.
4. Страуструп, Б. Принципы и практика использования C++ / Б. Страуструп – 2009 – 1238 с.
5. Ирвин, К. Язык ассемблера для процессоров Intel, 4-е издание.: Пер. с англ. – М.: Вильямс, 2005. – 912с.
6. Герберт, Ш. Справочник программиста по C/C++ / Шилдт Герберт. - 3-е изд. – Москва : Вильямс, 2003. – 429 с.

Приложение А

```

byte task getAnd(byte a, byte b) {
    response a&b;
}

int task getMul(int a, int b) {
    response a*b;
}

begin {
    cout("тест функции getAnd");
    def byte yesand = getAnd(4, 5);
    cout(yesand);

    cout("тест функции getMul");
    def int iv = getMul(200, 200);
    cout(iv);

    cout("тест операций (логических, унарных, арифметических)");
    def byte yesor = 3 | 5;
    def byte inversion = ~11;

    def byte az = 5d;
    def byte bz = 9;
    def byte incr = ++az;
    def byte decr = bz--;
    def int w;
    w=100/2;

    cout(yesor);
    cout(inversion);
    cout(incr);
    cout(decr);
    cout(w);

    cout("тест функции сравнения строк стандартной библиотеки");
    def str s1 = "aq";
    def str s2 = "a";
    def int result = strcmp(s1, s2);
    cout(result);

    cout("тест функции преобразования строки в число стандартной библиотеки");
    def str s3 = "123";
    def int res = atoi(s3);
    cout(res);

    cout("тест типы данных");
    def byte bb = 1011b;
    cout(bb);
    def symb symbol = 'Q';
    def str line = "it`s line";

```

```

cout(symbol);
cout(line);
cout("просто текст");

cout("тест условного оператора switch-is-any");
def byte sw = 1;
switch(sw) {
    is 2:{cout("two");}
    is 1:{cout("one");}
    is 3:{cout("three");}
    any:{cout("any");}
}

cout("тест условного оператора when-other");
def torf flag = 0;
when (inversion > 2) {
    flag = 1;
    cout(flag);
}
other {
    cout(flag);
}

cout("тест вложенных условных операторов");
def int val = 2;
switch(val) {
    is 2: {
        cout("val is 2");
        when(bz > 5) {
            cout("and bz > 5");
        }
    }
    any: { cout("other"); }
}

!!comment

cout("тест сложного выражения");
def byte clozh = (1+2)*2-2*2+val;!!test test
cout(clozh);

}

```

Листинг А.1 – Контрольный пример на языке LPA-2025

Приложение Б

```

struct Entry
{
    char lexema[LEXEMA_FIXSIZE];
    int sn;
    int idxTI;
};
struct lexTable
{
    int maxsize;
    int size;
    Entry* table;
};

```

Листинг Б.1 – Реализация таблицы лексем

```

enum IDDATATYPE { INT = 1, STR = 2, BYTE = 3, CH = 4, BOOL = 5};
enum IDTYPE { V = 1, F = 2, P = 3, L = 4, SF = 5, M = 6};

struct Entry
{
    int          idxfirstLE;
    int          line;
    char         id[ID_MAXSIZE];
    IDDATATYPE   iddatatype;
    IDTYPE       idtype;
    Entry* scope;
    union
    {
        int      vint;
        char vbyte;
        char vchar;
        bool vbool;
        struct
        {
            int len;
            char str[TI_STR_MAXSIZE - 1];
        } vstr[TI_STR_MAXSIZE];
    } value;
};

struct idTable
{
    int maxsize;
    int size;
    Entry* table;
};

```

Листинг Б.1 – Реализация таблицы идентификаторов

Приложение В

```
1   tfi(ti,ti){
2   rivi;
3   }
5   tfi(ti,ti){
6   rivi;
7   }
9   m{
10  p(l);
11  dti=i(l,l);
12  p(i);
14  p(l);
15  dti=i(l,l);
16  p(i);
18  p(l);
19  dti=lvl;
20  dti=~l;
22  dti=l;
23  dti=l;
24  dti=[i;
25  dti=i_;
26  dti;
27  i=lvl;
29  p(i);
30  p(i);
31  p(i);
32  p(i);
33  p(i);
35  p(l);
36  dti=l;
37  dti=l;
38  dti=C(i,i);
39  p(i);
41  p(l);
42  dti=l;
43  dti=T(i);
44  p(i);
46  p(l);
47  dti=l;
48  p(i);
49  dti=l;
50  dti=l;
51  p(i);
52  p(i);
53  p(l);
55  p(l);
56  dti=l;
57  h(i){
58  cl:{p(l);}
59  cl:{p(l);}
60  cl:{p(l);}
```

```

61  a:{p(l);}
62  }
64  p(l);
65  dti=l;
66  s(ibl){
67  i=l;
68  p(i);
69  }
70  e{
71  p(i);
72  }
74  p(l);
75  dti=l;
76  h(i){
77  cl:{
78  p(l);
79  s(ibl){
80  p(l);
81  }
82  }
83  a:{p(l);}
84  }
88  p(l);
89  dti=(lvl)vlvlvlvi;
90  p(i);
92  }

```

Листинг В.1 – Таблица лексем

id	datatype	idtype	Line	Scope	value
getAnd_1	BYTE	F	1	-	0
a_1	BYTE	P	1	getAnd	0
b_1	BYTE	P	1	getAnd	0
getMul_5	INT	F	5	-	0
a_5	INT	P	5	getMul	0
b_5	INT	P	5	getMul	0
begin_9	INT	M	9	-	0
L1_10	STR	L	10	begin	"
getAnd"					
yesand_11	BYTE	V	11	begin	0
L2_11	INT	L	11	-	4
L3_11	INT	L	11	-	5
L5_14	STR	L	14	begin	"
getMul"					
iv_15	INT	V	15	begin	0
L6_15	INT	L	15	-	200
L7_15	INT	L	15	-	200
L9_18	STR	L	18	begin	"
(, ,)"					
yesor_19	BYTE	V	19	begin	0
L10_19	INT	L	19	-	3
L11_19	INT	L	19	-	5
inversion_20	BYTE	V	20	begin	0

L12_20	INT	L	20	-	11
az_22	BYTE	V	22	begin	0
L13_22	INT	L	22	-	5
bz_23	BYTE	V	23	begin	0
L14_23	INT	L	23	-	9
incr_24	BYTE	V	24	begin	0
decr_25	BYTE	V	25	begin	0
w_26	INT	V	26	begin	0
L15_27	INT	L	27	-	100
L16_27	INT	L	27	-	2
L18_35	STR	L	35	begin	"
"					
s1_36	STR	V	36	begin	
L20_36	STR	L	36	begin	"aq"
s2_37	STR	V	37	begin	
L22_37	STR	L	37	begin	"a"
result_38	INT	V	38	begin	0
strcpr_38	BYTE	SF	38	begin	0
L24_41	STR	L	41	begin	"
"					
s3_42	STR	V	42	begin	
L26_42	STR	L	42	begin	"123"
res_43	INT	V	43	begin	0
atoli_43	BYTE	SF	43	begin	0
L28_46	STR	L	46	begin	"
"					
bb_47	BYTE	V	47	begin	0
L29_47	INT	L	47	-	11
symbol_49	SYMB	V	49	begin	' '
C31_49	SYMB	L	49	begin	'Q'
line_50	STR	V	50	begin	
L32_50	STR	L	50	begin	"it`s
line"					
L34_53	STR	L	53	begin	"
"					
L36_55	STR	L	55	begin	"
switch-is-any"					
sw_56	BYTE	V	56	begin	0
L37_56	INT	L	56	-	1
L38_58	INT	L	58	-	2
L40_58	STR	L	58	begin	"two"
L41_59	INT	L	59	-	1
L43_59	STR	L	59	begin	"one"
L44_60	INT	L	60	-	3
L46_60	STR	L	60	begin	"three"
L48_61	STR	L	61	begin	"any"
L50_64	STR	L	64	begin	"
when-other"					
flag_65	TORF	V	65	begin	0
L51_65	INT	L	65	-	0
L52_66	INT	L	66	-	2
L53_67	INT	L	67	-	1
L55_74	STR	L	74	begin	"

"					
val_75	INT	V	75	begin	0
L56_75	INT	L	75	-	2
L57_77	INT	L	77	-	2
L59_78	STR	L	78	begin	"val
is_2"					
L60_79	INT	L	79	-	5
L62_80	STR	L	80	begin	"and
bz>5"					
L64_83	STR	L	83	begin	"other"
L66_88	STR	L	88	begin	"
"					
clozh_89	BYTE	V	89	begin	0
L67_89	INT	L	89	-	1
L68_89	INT	L	89	-	2
L69_89	INT	L	89	-	2
L70_89	INT	L	89	-	2
L71_89	INT	L	89	-	2

Листинг В.2 – Таблица идентификаторов

Приложение Г

```

struct Rule                                //правило в грамматике Грейбах
{
    GRBALPHABET nn;
    int iderror;
    short size;
    struct Chain                            //цепочка(правая часть правила)
    {
        short size;
        GRBALPHABET* nt;
        Chain(){
            this->size = 0;
            this->nt = 0;};
        Chain(short psize, GRBALPHABET s, ...);
        char* getCChain(char* b);
        static GRBALPHABET T(char t){ return GRBALPHABET(t); };
        static GRBALPHABET N(char n){return -GRBALPHABET(n); };
        static bool isT(GRBALPHABET s) { return s > 0; };
        static bool isN(GRBALPHABET s) { return !isT(s); };
        static char alphabet_to_char(GRBALPHABET s)
        {return isT(s) ? char(s) : char(-s);};
    }*chains;
    Rule(){
        this->nn = 0x00;
        this->size = 0;}
    Rule(GRBALPHABET pnn,                //(нетерминал(< 0);
        int iderrore,
        short psize,
        Chain c, ...);
    char* getCRule(char* b, short nchain);
    short getNextChain(GRBALPHABET t, Rule::Chain& pchain, short
j);
};

```

Листинг Г.1 – Реализация структур Rule и Rule::Chain

```

struct Greibach                            //грамматика Грейбах
{
    short size;                            //количество правил
    GRBALPHABET startN;                    //стартовый символ
    GRBALPHABET stbottomT;                 //дно стека
    Rule* rules;                           //множество правил
    Greibach() { this->size = 0; this->startN = 0; this-
>stbottomT = 0; this->rules = 0; };
    Greibach(GRBALPHABET pstartN, GRBALPHABET pstbottomT, short
psize, Rule r, ...);
    short getRule(GRBALPHABET pnn, Rule& prule);
    Rule getRule(short n);
};

```

Листинг Г.2 – Реализация структуры Greibach

Приложение Д

```

0  всего строк 92, синтаксический анализ выполнен без ошибок
0  : S->tfi (F) {N}S
4  : F->ti, F
7  : F->ti
11 : N->rE;
12 : E->iM
13 : M->vE
14 : E->i
17 : S->tfi (F) {N}S
21 : F->ti, F
24 : F->ti
28 : N->rE;
29 : E->iM
30 : M->vE
31 : E->i
34 : S->m{N}
36 : N->pE;N
38 : E->l
41 : N->dtN
43 : N->i=E;N
45 : E->i (W)
47 : W->l, W
49 : W->l
52 : N->pE;N
53 : E->(E)
54 : E->i
57 : N->pE;N
59 : E->l
62 : N->dtN
64 : N->i=E;N
66 : E->i (W)
68 : W->l, W
70 : W->l
73 : N->pE;N
74 : E->(E)
75 : E->i
78 : N->pE;N
80 : E->l
83 : N->dtN
85 : N->i=E;N
87 : E->lM
88 : M->vE
89 : E->l
91 : N->dtN
93 : N->i=E;N
95 : E->~E
96 : E->l
98 : N->dtN
100 : N->i=E;N
102 : E->l
104 : N->dtN

```

```
106 : N->i=E;N
108 : E->l
110 : N->dtN
112 : N->i=E;N
114 : E->[E
115 : E->i
117 : N->dtN
119 : N->i=E;N
121 : E->iM
122 : M->_
124 : N->dti;N
128 : N->i=E;N
130 : E->lM
131 : M->vE
132 : E->l
134 : N->pE;N
135 : E->(E)
136 : E->i
139 : N->pE;N
140 : E->(E)
141 : E->i
144 : N->pE;N
145 : E->(E)
146 : E->i
149 : N->pE;N
150 : E->(E)
151 : E->i
154 : N->pE;N
155 : E->(E)
156 : E->i
159 : N->pE;N
160 : E->(E)
161 : E->l
164 : N->dtN
166 : N->i=E;N
168 : E->l
170 : N->dtN
172 : N->i=E;N
174 : E->l
176 : N->dtN
178 : N->i=E;N
180 : E->C(W)
182 : W->i,W
184 : W->i
187 : N->pE;N
189 : E->i
192 : N->pE;N
193 : E->(E)
194 : E->l
197 : N->dtN
199 : N->i=E;N
201 : E->l
203 : N->dtN
```

```

205 : N->i=E;N
207 : E->T(W)
209 : W->i
212 : N->pE;N
213 : E->(E)
214 : E->i
217 : N->pE;N
218 : E->(E)
219 : E->l
222 : N->dtN
224 : N->i=E;N
226 : E->l
228 : N->pE;N
229 : E->(E)
230 : E->i
233 : N->dtN
235 : N->i=E;N
237 : E->l
239 : N->dtN
241 : N->i=E;N
243 : E->l
245 : N->pE;N
246 : E->(E)
250 : N->pE;N
252 : E->i
255 : N->pE;N
256 : E->(E)
257 : E->l
260 : N->pE;N
261 : E->(E)
262 : E->l
265 : N->dtN
267 : N->i=E;N
269 : E->l
271 : N->h(i){K}N
276 : K->cl:{N}K
280 : N->pE;
282 : E->l
286 : K->cl:{N}K
290 : N->pE;
291 : E->(E)
296 : K->cl:{N}K
300 : N->pE;
301 : E->(E)
302 : E->l
306 : K->a:{N}
309 : N->pE;
310 : E->(E)
311 : E->l
316 : N->pE;N
317 : E->(E)
318 : E->l
321 : N->dtN

```

```

323 : N->i=E;N
325 : E->l
327 : N->s (B) {N}e{N}N
329 : B->ibl
334 : N->i=E;N
336 : E->l
338 : N->pE;
340 : E->i
346 : N->pE;
348 : E->i
352 : N->pE;N
354 : E->l
357 : N->dtN
359 : N->i=E;N
361 : E->l
363 : N->h(i) {K}N
368 : K->cl:{N}K
372 : N->pE;N
373 : E->(E)
374 : E->l
377 : N->s (B) {N}
379 : B->ibl
384 : N->pE;
385 : E->(E)
386 : E->l
391 : K->a:{N}
394 : N->pE;
396 : E->l
401 : N->pE;N
403 : E->l
406 : N->dtN
408 : N->i=E;N
410 : E->(E)M
411 : E->lM
412 : M->vE
413 : E->l
415 : M->vE
416 : E->lM
417 : M->vE
418 : E->lM
419 : M->vE
420 : E->lM
421 : M->vE
422 : E->i
424 : N->pE;
425 : E->(E)
426 : E->i

```

Листинг Д.1 – Дерево разбора

Приложение Е

```

map<char, int> Priorities = {
    {'(', 0},
    {')', 0},
    {'~', 4},
    {'{', 4},
    {'}', 4},
    {'[', 4},
    {']', 4},
    {'^', 4},
    {'_', 4},
    {'*', 3},
    {'/', 3},
    {'+', 2},
    {'-', 2},
    {'>', 1},
    {'<', 1},
    {'&', 1},
    {'|', 1},
    {',', 1},
    {':', 1},
};

namespace PN {
    bool polishNotation(int i, LA::LEX& lex){
        std::stack<LT::Entry> stack;
        std::queue<LT::Entry> queue;

        LT::Entry aggregate_symbol;
        aggregate_symbol.idxTI = -1;
        aggregate_symbol.lexema[0] = '#';
        aggregate_symbol.sn = lex.lexTable.table[i].sn;
        LT::Entry function_symbol;
        function_symbol.idxTI = LT_TI_NULLIDX;
        function_symbol.lexema[0] = '@';
        function_symbol.sn = lex.lexTable.table[i].sn;
        int idx = LT_TI_NULLIDX;
        int lexem_counter = 0;
        int parm_counter = 0;
        int lexem_position = i;
        char* buf = new char[21];
        bool is_task = false;
        for (i; lex.lexTable.table[i].lexema[0] !=
LEX_SEMICOLON; i++, lexem_counter++) {
            switch (lex.lexTable.table[i].lexema[0]) {
                case LEX_ID:
                case LEX_LITERAL:
                    if
(lex.idTable.table[lex.lexTable.table[i].idxTI].idtype == IT::F)
{
                        is_task = true;
                        idx = lex.lexTable.table[i].idxTI;}
                    else {

```

```

        if (is_task)
            parm_counter++;
        queue.push(lex.lexTable.table[i]);
    }
    continue;
case TILDE:
case INC:
case DEC:
case POST_INC:
case POST_DEC:
case LEFTTHESIS:
    stack.push(lex.lexTable.table[i]);
    continue;
case RIGHTTHESIS:
    while (stack.top().lexema[0] != LEFTTHESIS){
        queue.push(stack.top());
        stack.pop();
        if (stack.empty()) {
            delete[] buf;
            return false;}
    }
    stack.pop();
    if (is_task) {
        function_symbol.idxTI = idx;
        idx = LT_TI_NULLIDX;
        LT::Entry tempFunc = function_symbol;
        queue.push(tempFunc);
        _itoa_s(parm_counter, buf, 21, 10);
        LT::Entry countEntry;
        countEntry.idxTI = LT_TI_NULLIDX;
        countEntry.lexema[0] = buf[0];
        countEntry.sn = function_symbol.sn;
        queue.push(countEntry);
        parm_counter = 0;
        is_task = false;}
    continue;
case LEX_OPERATION:{
    char currentOp =
lex.lexTable.table[i].lexema[1];
    while (!stack.empty()) {
        char topOp = stack.top().lexema[0];
        if (topOp == LEX_OPERATION) topOp =
stack.top().lexema[1];
        if (topOp == '(') break;
        if (Priorities[topOp]
Priorities[currentOp]) {
            queue.push(stack.top());
            stack.pop();
        }
        else {break;}
    }
    stack.push(lex.lexTable.table[i]);
    continue;}

```

```

        case LEX_STRCPR:
        case LEX_ATOII:
            is_task = true;
            idx = lex.lexTable.table[i].idxTI;
            continue;}
    }
    while (!stack.empty()) {
        if (stack.top().lexema[0] == LEFTTHESIS ||
stack.top().lexema[0] == RIGHTTHESIS) {
            delete[] buf;
            return false;
        }

        queue.push(stack.top());
        stack.pop();
    }

    while (lexem_counter != 0) {
        if (!queue.empty()) {
            lex.lexTable.table[lexem_position++] =
queue.front();
            queue.pop();
        }
        else
            lex.lexTable.table[lexem_position++] =
aggregate_symbol;
        lexem_counter--;
        for (int k = 0; k < lexem_position; k++) {
            if (lex.lexTable.table[k].lexema[0] ==
LEX_OPERATION || lex.lexTable.table[k].lexema[0] == LEX_LITERAL)
            {
                if (lex.lexTable.table[k].idxTI != -1)

                lex.idTable.table[lex.lexTable.table[k].idxTI].idxfirstLE =
k;
            }
            delete[] buf;
            return true;
        }
    }
    bool startPolish(LA::LEX& lex) {
        bool result = false;
        for (int i = 0; i < lex.lexTable.size; i++) {
            if (lex.lexTable.table[i].lexema[0] == '=') {
                result = polishNotation(i + 1, lex);
                if (!result) {return false;}
            }
        }
        return true;
    }
}

```

Листинг Е.1 – Реализация преобразования в ПОЛИЗ

Приложение Ж

```
.586P
.model flat, stdcall
includelib libcrt.lib
includelib kernel32.lib
includelib "D:\bstu\3sem\lpa2\lpa\LPA-2025\Debug\LIB.lib"
ExitProcess PROTO : DWORD
SetConsoleCP PROTO : DWORD
SetConsoleOutputCP PROTO : DWORD
strcmp PROTO : DWORD, : DWORD
atoi PROTO : DWORD
writestr PROTO : DWORD
writeint PROTO : SDWORD
writebool PROTO : BYTE
writechar PROTO : BYTE
.stack 4096
.const
L1 DB "тест функции getAnd", 0 ; str
L2 SDWORD 4 ; int (4 bytes)
L3 SDWORD 5 ; int (4 bytes)
L5 DB "тест функции getMul", 0 ; str
L6 SDWORD 200 ; int (4 bytes)
L7 SDWORD 200 ; int (4 bytes)
L9 DB "тест операций(логических,унарных,арифметических)", 0 ; str
L10 SDWORD 3 ; int (4 bytes)
L11 SDWORD 5 ; int (4 bytes)
L12 SDWORD 11 ; int (4 bytes)
L13 SDWORD 5 ; int (4 bytes)
L14 SDWORD 9 ; int (4 bytes)
L15 SDWORD 100 ; int (4 bytes)
L16 SDWORD 2 ; int (4 bytes)
L18 DB "тест функции сравнения строк стандартной библиотеки", 0
L20 DB "aq", 0 ; str
L22 DB "a", 0 ; str
L24 DB "тест функции преобразования строки в число стандартной
библиотеки", 0 ; str
L26 DB "123", 0 ; str
L28 DB "тест типы данных", 0 ; str
L29 SDWORD 11 ; int (4 bytes)
C31 BYTE 81 ; symb
L32 DB "it`s line", 0 ; str
L34 DB "просто текст", 0 ; str
L36 DB "тест условного оператора switch-is-any", 0 ; str
L37 SDWORD 1 ; int (4 bytes)
L38 SDWORD 2 ; int (4 bytes)
L40 DB "two", 0 ; str
L41 SDWORD 1 ; int (4 bytes)
L43 DB "one", 0 ; str
L44 SDWORD 3 ; int (4 bytes)
L46 DB "three", 0 ; str
L48 DB "any", 0 ; str
L50 DB "тест условного оператора when-other", 0 ; str
```

```

L51 SDWORD 0 ; int (4 bytes)
L52 SDWORD 2 ; int (4 bytes)
L53 SDWORD 1 ; int (4 bytes)
L55 DB "тест вложенных условных операторов", 0 ; str
L56 SDWORD 2 ; int (4 bytes)
L57 SDWORD 2 ; int (4 bytes)
L59 DB "val is 2", 0 ; str
L60 SDWORD 5 ; int (4 bytes)
L62 DB "and bz>5", 0 ; str
L64 DB "other", 0 ; str
L66 DB "тест сложного выражения", 0 ; str
L67 SDWORD 1 ; int (4 bytes)
L68 SDWORD 2 ; int (4 bytes)
L69 SDWORD 2 ; int (4 bytes)
L70 SDWORD 2 ; int (4 bytes)
L71 SDWORD 2 ; int (4 bytes)
.data
yesand SDWORD 0 ; int
iv SDWORD 0 ; int
yesor SDWORD 0 ; int
inversion SDWORD 0 ; int
az SDWORD 0 ; int
bz SDWORD 0 ; int
incr SDWORD 0 ; int
decr SDWORD 0 ; int
w SDWORD 0 ; int
s1 DWORD 0 ; str
s2 DWORD 0 ; str
result SDWORD 0 ; int
s3 DWORD 0 ; str
res SDWORD 0 ; int
bb SDWORD 0 ; int
symbol BYTE 0 ; symb
line DWORD 0 ; str
sw SDWORD 0 ; int
flag BYTE 0 ; boolean
val SDWORD 0 ; int
clozh SDWORD 0 ; int
.code
FgetAnd PROC uses ebx ecx edi esi, a : SBYTE, b : SBYTE
movsx eax, a
mov a, al
movzx eax, a
mov al, a
and al, b
mov b, al
ret
FgetAnd ENDP
FgetMul PROC uses ebx ecx edi esi, a : DWORD, b : DWORD
mov eax, a
mov ecx, a
imul ecx, b
mov b, eax

```

```

ret
FgetMul ENDP
main PROC
Invoke SetConsoleCP, 1251
Invoke SetConsoleOutputCP, 1251
push offset L1
CALL writestr
push L2
push L3
call FgetAnd
push eax
pop eax
mov yesand, eax
mov eax, yesand
push eax
CALL writeint
push offset L5
CALL writestr
push L6
push L7
call FgetMul
push eax
pop eax
mov iv, eax
mov eax, iv
push eax
CALL writeint
push offset L9
CALL writestr
push L10
push L11
pop ebx
pop eax
or eax, ebx
push eax
pop eax
mov yesor, eax
push L12
pop eax
not eax
push eax
pop eax
mov inversion, eax
mov eax, L13
mov az, eax
mov eax, L14
mov bz, eax
push az
pop eax
inc eax
mov byte ptr [az], al
push eax
pop eax

```

```

mov incr, eax
push bz
pop eax
mov ebx, eax
dec eax
mov byte ptr [bz], al
mov eax, ebx
push eax
pop eax
mov decr, eax
push L15
push L16
pop ebx
pop eax
cdq
idiv ebx
push eax
pop eax
mov w, ea
mov eax, yesor
push eax
CALL writeint
mov eax, inversion
push eax
CALL writeint
mov eax, incr
push eax
CALL writeint
mov eax, decr
push eax
CALL writeint
mov eax, w
push eax
CALL writeint
push offset L18
CALL writestr
mov eax, OFFSET L20
mov [s1], eax
mov eax, OFFSET L22
mov [s2], eax
push s1
push s2
call strcpr
push eax
pop eax
mov result, eax
mov eax, result
push eax
CALL writeint
push offset L24
CALL writestr
mov eax, OFFSET L26
mov [s3], eax

```

```

push s3
call atoli
push eax
pop eax
mov res, eax
mov eax, res
push eax
CALL writeint
push offset L28
CALL writestr
mov eax, L29
mov bb, eax
mov eax, bb
push eax
CALL writeint
movsx eax, C31
mov symbol, al
mov eax, OFFSET L32
mov [line], eax
push eax
movzx eax, symbol
push eax
CALL writechar
pop eax
push line
CALL writestr
push offset L34
CALL writestr
push offset L36
CALL writestr
mov eax, L37
mov sw, eax
movzx eax, byte ptr [sw]
cmp eax, 2
jne Switch_1_Next_0
push offset L40
CALL writestr
jmp Switch_1_EXIT
Switch_1_Next_0:
cmp eax, 1
jne Switch_1_Next_1
push offset L43
CALL writestr
jmp Switch_1_EXIT
Switch_1_Next_1:
cmp eax, 3
jne Switch_1_Next_2
push offset L46
CALL writestr
jmp Switch_1_EXIT
Switch_1_Next_2:
push offset L48
CALL writestr

```

```

jmp Switch_1_EXIT
Switch_1_EXIT:
push offset L50
CALL writestr
mov eax, L51
mov flag, al
mov eax, inversion
mov ebx, L52
cmp eax, ebx
jg If_True_2
jmp If_Else_2
If_True_2:
mov eax, L53
mov flag, al
push eax
movzx eax, flag
push eax
CALL writebool
pop eax
jmp If_Exit_2
If_Else_2:
push eax
movzx eax, flag
push eax
CALL writebool
pop eax
If_Exit_2
push offset L55
CALL writestr
mov eax, L56
mov val, eax
mov eax, val
cmp eax, 2
jne Switch_4_Next_0
push offset L59
CALL writestr
mov eax, bz
mov ebx, L60
cmp eax, ebx
jg If_True_5
jmp If_Else_5
If_True_5:
push offset L62
CALL writestr
jmp If_Exit_5
If_Else_5:
If_Exit_5:
jmp Switch_4_EXIT
Switch_4_Next_0:
push offset L64
CALL writestr
jmp Switch_4_EXIT
Switch_4_EXIT:

```

```
push offset L66
CALL writestr
push L67
push L68
pop ebx
pop eax
add eax, ebx
push eax
push L69
pop ebx
pop eax
imul eax, ebx
push eax
push L70
push L71
pop ebx
pop eax
imul eax, ebx
push eax
pop ebx
pop eax
sub eax, ebx
push eax
push val
pop ebx
pop eax
add eax, ebx
push eax
pop eax
mov clozh, eax
mov eax, clozh
push eax
CALL writeint
push -1
call ExitProcess
main ENDP
end main
```

Листинг Ж.1 – Результат генерации исходного кода в ассемблер